**Citect Pty Ltd** ABN 88 001 158 854
3 Fitzsimons Lane Gordon 2072 NSW Australia
PO Box 174 Pymble 2073 NSW Australia
Tel 61 2 9496 7300 Fax 61 2 9496 7399 www.citect.com

CitectSCADA

**v5.4x**

# MOSCAD driver, User Information and Design

# Driver Spec Version History

| Author | Date | Comment |
|---|---|---|
| Stephen Burman | 21/10/97 | |
| Stephen Burman | 24/2/98 | |
| Stephen Burman | 21/3/98 | |
| David Rossini | 18/05/98 | |
| David Rossini | 21/09/98 | Enhancements for release of V2.01b1 |
| David Rossini | 22/09/98 | Technical Enhancements for release of V2.01b1 |
| David Rossini | 14/04/99 | Enhancements for release of V2.01b2 |
| Lin Gu | 10/11/00 | Modified for V3.00.00.001 B1 |
| JaeWoo Kim | 29/11/01 | Modified for V3.01.03.002 B1 (Refer to the Appendix) |
| Ben Liquete | 08/03/02 | Updated User Interface to include info re: Gateway Redundancy |
| Paul Nguyen/Lin Gu | 2/09/2002 | Gateway redundancy |
| Greg Roberts | 22/10/02 | Updated to include Gentronics changes in Holland by David Rossini, document to be reviewed by David when he returns.<br>Information valid for version 3.2.x.x |
| | | |

# Contents


**2 TARGET DEVICE(S) AND PROTOCOL 7**

**2.1 Introduction 7**

**2.2 Device Manufacturer 7**

**2.3 Device Definition 7**

**2.4 Communications Method 7**

**2.5 Communications/Hardware Configuration 8**

2.5.1 Wiring Diagrams 8

2.5.2 I/O Device Settings 8

2.5.3 Software Setup 8

There is a MOSCAD Programming Toolbox available to configure gateways and another to configure RTUs. The computer running the Toolbox uses an RS-232 connection, and can be connected to either a gateway or an RTU. 8

2.5.3.1 Gateway Software Configuration 8

2.5.3.2 Gateway Mode of Operation Configuration 9

2.5.3.3 RTU Software Configuration 9

**2.6 Special Requirements 11**

**2.7 Maximum Request Length 12**

**3 PROTOCOL REQUIREMENTS 13**

**3.1 Introduction 13**

**3.2 Initialising the Driver 14**

**3.3 Initialising the Board 14**

**3.4 Initialising the Port 14**

**3.5 Initialising the IO Device 14**

**3.6 IO Device Online Test 14**

**3.7 Internal Data structures for storage of cached data within the driver 15**

**3.8 Cached RTU table format 17**

**3.9 Request Queue Structures 17**

**3.10 State Flow Description 17**

3.10.1 Cache update strategy 18

3.10.2 Checking Cache freshness 19

3.10.3 Handling Timeouts 19

**3.11 Message Structure 19**

4.11.1 MDLC Buffer Format 19

3.11.1.1 MDLC Buffer Header 19

3.11.1.2 Data Blocks Structure 20

3.11.2 BASIC SCADA SERVICES - DATA BLOCKS & PROTOCOLS 22

3.11.2.1 Poll Data Protocol: 24
3.11.2.2 Poll Data with COS (Change Of State) Protocol: 25
3.11.2.3 Burst Data Transfer Protocol: 26
3.11.2.4 RTU Event Data Transfer Protocol: 26
3.11.2.5 Send Data Block (for Send Data (DB MAP) Applications) 27
3.11.3 GATEWAY MANAGEMENT - DATA BLOCKS & PROTOCOLS 28
3.11.3.1 Connect Channel Data Block 28
3.11.3.2 Disconnect Channel Data Block 29
3.11.3.3 Close Client Data Block 29
3.11.3.4 Set Gateway Mode Data Block 30
3.11.3.5 Get Gateway Mode Data Block 30
3.11.3.6 Get Link Status Data Block 31
3.11.4 Gateway Issues 31
3.11.5 EXTENDED SCADA SERVICES - DATA BLOCKS & PROTOCOLS 32
3.11.5.1 Broadcast Data Protocol 32
3.11.5.2 Get RTU's Date & Time Data Protocol 33
3.11.5.3 Set RTU's Date & Time Data Protocol 34
3.11.5.4 Sync RTUs Time Data Protocol 35

**4 USER INTERFACE 36**

**4.1 Introduction 36**

**4.2 Driver Name 36**

**4.3 Boards Form 36**

**4.4 Ports Form 36**

**4.5 IO Devices Form 37**

1.1.1 Address : 37

4.5.1 # NameOfDevice 37

**4.6 Pulldown lists Help 37**

**4.7 IO Device Variable Types 38**

4.7.1 Formats and types 38

4.7.2 Event data types (alarm and trend) 39

4.7.3 Cicode functions to access timestamped event data 41

4.7.4 MOSCAD.dbf Entries 43

**4.8 PROTDIR.DBF 43**

**4.9 Parameters and INI options 45**

4.9.1 Standard Parameters 45

5.9.2 Driver Specific Parameters 45

**4.10 Configuration file 46**

4.10.1 Format of configuration file 46

4.10.2 Sample of a configuration file 47

4.10.3 Sample of a CITECT.INI file 48

**4.11 Using Gateway Redundancy 49**

**4.12 Remapping 50**

**4.13 Hints and Tips 50**

**5 APPENDIX 52**

**5.1 Notes from Field Work for version 3.2.x.x 52**

5.1.1 Driver Issues 52

5.1.2 Additional information regarding gateway operation 52

5.1.3 Comments on code changes to the generic version (“AS31”) 53

5.1.4 Status of driver version 54

5.1.5 Possible Future work 54

5.1.6 Notes on temporary “freeze” at startup when critical RTU not contactable 54

**5.2 MOSCAD Driver Modification List for V30103002b1 58**

5.2.1 C_BIT Problem 58

5.2.2 C_BIT bit swap operation 58

5.2.3 Timeout Value Long 58

5.2.4 POLLING PROCESS OPERTION 58

5.2.5 Channel Data Reading problem on start time 59

# 2 Target Device(s) and Protocol

## 2.1 Introduction

This section defines the types of I/O Devices that are targeted by this driver.

## 2.2 Device Manufacturer

Motorola Communications Group

## 2.3 Device Definition

The device is a hardware system, consisting of a number of RTUs (Remote Terminal Unit) which communicate to the outside world via a Gateway. The Gateway abstracts communications so that all RTUs are communicated with using the same Application Programming Interface (API) calls regardless of the communication path (e.g. leased line, radio modem e.t.c).

The Gateway communicates to RTUs using the proprietary MDLC protocol over RS-485 and radio modem. The gateway has 3 serial ports for communications with the RTUs or the programming toolbox. Also the gateway has two network connectors (10Base-T and AUI), either of which can be used.

Communication with the gateway is possible using TCP/IP and a standard ethernet card.

Each RTU consists of a CPU module and various IO and communication modules.

## 2.4 Communications Method

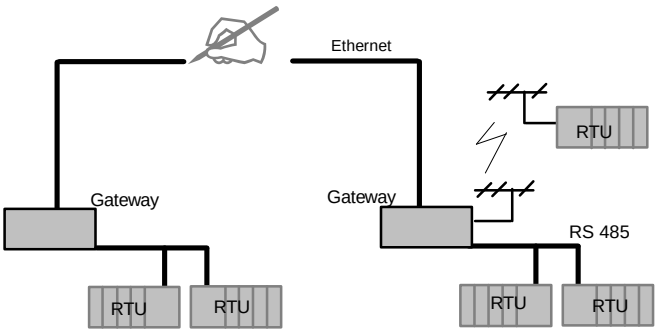
The Gateway sits on an ethernet network and communicates to the SCADA system using the TCP/IP protocol.

The Gateway can theoretically communicate to a total of 32768 RTUs at one time. The practical maximum is not specified but section 7.1 in the MOSCAD developers guide mentions that a minimum of 250 RTUs should be supported per gateway, and at least 16 gateways should be supported. Therefore the standard skeleton driver constants must MAX_UNIT must be at least 250, and MAX_CHANNEL must be at least 16.

From the MDLC API users point of view, each RTU consists of a user configurable number of tables (up to 128), each of user configurable dimensions (up to a maximum of 250 rows by 8 columns), and user configurable makeup (e.g. bit, bit, float or int, float, int). The columns are made up of only one data type.

The communication philosophy is client / server. A SCADA system is considered the **client** and the gateway the server. A client communicates to the server using one or more **channels** (TCP/IP sockets). It is recommended to use a different channel for each type of communications (commands, poll data, burst data and management data).

## 2.5 Communications/Hardware Configuration




### 2.5.1 Wiring Diagrams

Standard ethernet cabling for connecting Citect to Gateway.

Diagnostics cable for use with ToolBox Software:

*Adaptor Cable Pinouts*

| *Colour* | *RJ45* | *DB25* | *DB9* |
|---|---|---|---|
| Blue | 1 | 5 | 8 |
| Orange | 2 | 6 | 6 |
| Black | 3 | 8 | 1 |
| Red | 4 | 4 | 7 |
| Green | 5 | 7 | 5 |
| Yellow | 6 | 20 | 4 |
| Brown | 7 | 2 | 3 |
| Grey | 8 | 3 | 2 |

### 2.5.2 I/O Device Settings

Communications set up is a function of windows. The network card must be installed with support for TCP/IP loaded and the local IP address selected.

### 2.5.3 Software Setup

Set up the local IP address using standard Windows network installation tools from control panel.

***There is a MOSCAD Programming Toolbox available to configure gateways and another to configure RTUs. The computer running the Toolbox uses an RS-232 connection, and can be connected to either a gateway or an RTU.***

#### *2.5.3.1 Gateway Software Configuration*

The Gateway toolbox is used to modify the following four groups of parameters.

?? TCP/IP Driver (IP address)

?? MDLC Gateway configuration (MDLC Gateway, Site-id and Link-ids)

?? MDLC Network configuration (system wide MDLC routers)

?? MDLC Site table (system wide Site-id and Link-ids)

#### *2.5.3.2 Gateway Mode of Operation Configuration*

?? If 1 Gateway is being used, the user should set the Gateway startup mode as “standalone”.

?? For Gateway redundancy, the user should setup 1 Gateway as “Primary” mode and the second Gateway as “Standby” mode. This can be set using the Gateway ToolBox program.

?? For “Primary” gateway operation, the user should set the gateway startup mode as “GW1”. For “Standby” gateway operation, the user should set the gateway startup mode as “GW2”.

#### *2.5.3.3 RTU Software Configuration*

The Gateway toolbox is used to modify the following four groups of parameters.

?? SiteId (RTU address)

?? RTU Site configuration (I/O cards, ports, etc)

?? RTU Database configuration (RTU tables of data)

?? RTU User Application configuration (RTU ladder logic)

The RTU Database configuration is used to hold either real I/O or user variables.
The RTU User Application configuration (ladder logic) is used to process variable and I/O data for monitoring and control purposes.

The amount of User Application configuration required is dependant on the communication strategy that is to be implemented. Sample RTU configuration is also included with (and is required for) the sample Citect project for this driver.

All data polled by Citect, no data Time Stamped.
Only user tables, for the data to be polled by Citect, need be configured in the RTU. The type configuration files (refer to Section 3.6), must also be edited to reflect these user tables.
In this case (the simplest possible case), no RTU User Application configuration (ladder logic) is required.

Unsolicted Time Stamped Digital Input event transmitted to Citect:
A user Time Stamped Digital Input table (this user table will be termed the TSDI table) must be configured, and linked to the required Digital Inputs. The column should be of type “TgDI” and should be linked as an “event” type, so that the RTU will place the Time Stamped Digital Input events onto the RTU event queue. Ladder Logic must be configured to extract a Time Stamped Digital Input event from the internal queue and place it into the system Primary Event Table. The state and time stamp data is then copied from the Primary Event Table to another user table (this user table will be termed the Buffered TSDI table). This updated row of the user table is then “burst” back to Citect. Refer to “MOSCAD Programming ToolBox – Special Functions” manual, Section “Event Driven

Software", for further background on the Primary Event table. Refer to "MOSCAD/MOSCAD-L Programming ToolBox for Windows (Version 5.0) – Users Guide" manual, Section "User Defined MDLC Communication", Subsections "Data Burst Table" and "Burst Function" (Pages 309 and 310 in Version 5.0) for further background on "Bursting" data to Citect.
Note that in order to utilise the Burst function, the table holding the data must have a symbol defined for it, hence one must be allocated when creating the Buffered TSDI table. The Burst function relies on a system table, the "Site Table", to define the Site ID and Link ID of the Gateway, so that data can be "burst" to Citect via the gateway.
The data column to be copied from the Primary Event table to the Buffered TSDI table, is "___Bit". This column should be configured in the Buffered TSDI table as type "Bit", and defined in the type configuration file (refer to Section 3.6) as "Bit". The two timestamp columns in the Primary Event table, that have to be copied to the Buffered TSDI table, are "TmMost" and "TmLeas". Both timestamp columns in the Buffered TSDI table are to be configured as type Real, but must be defined in the type configuration file as TS1 for the column copied from "TmMost", and TS2 for the column copied from "TmLeas".
Note, if a Digital Input is being used in a Time Stamped Digital Input table, then it should not be duplicated in a standard Digital Input table. This may result in a value being shown in a timestamped digital alarm and a different (older) value still being shown in the Digital Input tag, as the Digital Input tag will not be updated with the Time Stamped Digital Input data.

Application Time Stamping of data
If the project is utilising application timestamping of data, then the application time stamping will be conducted every scan cycle ( or second cycle, etc), and will be used to indicate the time at which this RTU scan acquired this data. User timestamping will only be accurate, at most, to the resolution of the RTU scan time. In comparison, the system timestamping of Digital Inputs is accurate to the millisecond, however Hardware timestamping places a large load on the RTU and should be used sparingly. The application timestamp will indicate the most recent time at which data has been acquired for this point, which may be a result of the RTU "bursting" an unsolicted response back to Citect, or the RTU responding to a Citect poll.

Application Time Stamping of individual data elements
Every column of data that is to be individually application time stamped is to be followed by two columns reserved for holding the time stamp data. Both timestamp columns are configured in the RTU as type Real, and are defined in the type configuration file (refer to Section 3.6) as TS1 for the column immediately following the data column, and TS2 for the next column. The content of these two columns is detailed in the manual "MOSCAD Programming ToolBox – Special Functions" , section "Event Driven Software", under the heading "Event Driven Mechanism". This section is detailing the Primary Event Table, used in the retrieval of Time Stamped Digital Events, but the Primary Event Table time stamp columns (listed as TmMost and TmLeas) can be set to the current Date/Time by calling the "Time" function in the users application code. These two columns are then copied to the required table locations to achieve user timestamping of data. The first timestamp column is copied from TmMost, and the second timestamp column is copied from TmLeas.

| TmMost | | | |
|---|---|---|---|
| Day | Month | Year | Hour |
| 1 Byte | 1 Byte | 1 Byte | 1 Byte |
| 1-31 | 1-12 | 0-99 | 0-23 |
| | | | |

| TmLeas | | |
|---|---|---|
| Minute | Second | Millisecond |
| 1 Byte | 1 Byte | 2 Bytes |
| 0-59 | 0-59 | 0-999 |
| | | |

User Time Stamping of complete rows
As an alternative to user timestamping individual data elements, a complete row can be user timestamped. This is achieved by utilising the first two columns of a table as timestamp columns (ie both defined in table as "Real", and defined in type configuration files as TS1 for the first column and TS2 for the second column). All data in that row will be time stamped with the Date/Time of this timestamp.

Interpretation of Hardware Timestamps
A decision has to be made whether a read to a TimeStamp tag of a TSDI is to represent the time at which an event last occurred for this point, or the last time at which data was acquired for this point.

The manner in which a timestamp (associated with a TSDI) is to be interpreted, will determine if the Buffered TSDI table needs to have the timestamp data continually updated by application timestamping, or if the timestamp is only updated by the hardware timestamping of the next event. In the first case, when the Time Stamped Digital Input event is being “burst” from this table, and in subsequent polls of this table, these timestamp columns will hold the time of the most recent event for this Digital Input. In the second case, the timestamp columns would be continually updated by application timestamping, in addition to the initial hardware timestamping, so that the timestamp columns will hold the time at which the data was acquired.
Note that, in the second case, the millisecond resolution of the hardware timestamping will be overridden by the scan time resolution of the application timestamping. However, this lower resolution will only be utilised in the polling algorithm and will only be used to update graphics; the alarm system will utilise the initial timestamp, with its millisecond resolution, that has been “burst” to Citect. The advantage of the second case is that the timestamps for the hardware based Time Stamped Digital Inputs will have the identical functionality of the application based timestamped data, ie the timestamp relates to the time at which the data was acquired. In this way, a tag read to the timestamp of this data will display the last point in time at which this data was acquired, either through unsolicited response or polling. This mechanism will also ensure that a timestamp value is obtained for a timestamp tag read, even if an event has not yet occurred since Citect startup.
If TimeStamp tags (that is reading the TimeStamp value) will not be used in the project for these points, then application timestamping of these hardware timestamped points is not required.

Unsolicted transmission of Change Of State data:
Ladder Logic must be configured to test the COS flag on each row of each table, and if the flag indicates that an element in that row has changed its value, then the row must be “burst” back to Citect. Refer to “MOSCAD/MOSCAD-L Programming ToolBox for Windows (Version 5.0) – Users Guide” manual, Section “User Defined MDLC Communication”, Subsections “Central To RTU Data Transfer” (Page 293 in Version 5.0) for further background on the COS flag.
Note that in order to utilse the COS flag, the table holding the data must have a name defined for the flag, and hence one must be configured by editing that table.

## 2.6 Special Requirements

We will need to ship the Winsock.DLL or alert users to the need for it.

The following information concerns the usage of the MOSCAD MDLC API.

Before running the driver make sure that the configuration files ("<mdlc_par>.cfg”, where mdlc_par is your file name. The path of this file must be set up in Citect.ini file as well. Refer to section 5.10) is present in the local directory and properly describe your MOSCAD system. To work properly, the API routines need information about the MOSCAD system and about the RTU types (communication table format). This information must be provided via two ASCII files:

<mdlc_par>.cfg - This file describes the format of the different existing RTU types (and future but currently non-existing RTU types) and type of each RTU in the MOSCAD system. There is a description of the communication tables format for each of the RTU types. The format defines the number of columns in a table, and the type of each column. A column can be either “Bit”, “Val”, “Float”, “C_Bit”, “TS1” or “TS2”; with Bit being for digitals, Val for 16 bit integers, Float for reals, C_Bit for eight columns of bits being compressed into one byte, and TS1 and TS2 used collectively for Time Stamping. Note that a TS1 column must be immediately followed by a TS2 column.
In this driver, there are two methods of timestamping data: individually or on a “per row” basis. To individually timestamp a data element, the data column in the table should be immediately followed by a TS1 and a TS2 column to hold the timestamp for that data element ( eg Bit TS1 TS2). In order to collectively timestamp an entire row, the first column should be of type TS1 and the second column should be of type TS2, all subsequent columns in the row will be timestamped by this value.

## 2.7 Maximum Request Length

2048 bits

# 3 Protocol Requirements

## 3.1 Introduction

Individual data tags for an RTU are addressed by table number, then row, then column.

A data tag can be one of the three following types:

1) Discrete (single bit flag)
2) Integer value (16-bit signed variable)
3) Real value (32-bit floating point variable), complies with IEEE 754.
4) Compressed Digital (eight columns of bits, compressed into a byte)

There are four other tag types available:

1) TimeStamp for a data tag (refer to Section 3.6 and Section 5.7.1 for usage details)
2) RTU Time
3) RTU Status
4) Synchromise RTU Time with Citect
5) Force Poll of RTU
6) Force COS Poll of RTU
7) Broadcast to multiple RTUs

This driver will support four types of data transfer

A) The transmission and reception of polled data
B) The transmission and reception of commands.
C) The reception of burst data. This mode of operation will allow events (such as alarms or event based trends) to be transferred from RTUs. This stream of data is handled via cicode.
D) The transmission and reception of management data.

As discussed in section 3.4 communication data can be segregated in separate streams using channels. Further, the gateway can handle a maximum of 20 pending requests. A MOSCAD channel can either be a SEND, RECEIVE, REGULAR (both send and receive) or BURST mode channel.

There is no indication in the documentation of how many MOSCAD channels (not Citect driver channels) can be opened at one time.

The following comment about channel allocation comes from section 3-5 of the MOSCAD developers guide (included with the development disk):

"If applicable, it is recommended that the SCADA Driver open dedicated channels for each of the following group of tasks:
· management task (set/get Gateway mode, ping)
· data acquisition task
· spontaneous data receiving task
· command"

The driver design will meet this goal by opening the following channels to each gateway:

- One channel will be dedicated to burst mode data reception.
- One channel will be dedicated to sending poll requests and receiving poll data.
- One channel will be dedicated to sending and receiving commands.
- One channel will be dedicated to sending and receiving management data.

There is no indication in the developers documentation that there are efficiency gains from using any more channels than indicated above. There is no indication of how many channels it is possible to set up.

It is suggested that each process using the MDLC API should be a separate client. The driver will be therefore be one client.

For this driver, an RTU will map to a Citect IO device, and a gateway will map to a Citect port (ie Citect channel). The boards form can be thought of as mapping to an ethernet card with the proviso that only one ethernet card is supported, and that no configuration is possible for this card.

The driver will support the built in gateway redundancy provided by the MDLC API. From the users point of view this means that a backup gateway can be configured in the ports form, for the primary gateway, as a special option. A backup gateway should not be entered as a separate ports form entry.

## 3.2 Initialising the Driver

No special driver initialisation will take place

## 3.3 Initialising the Board

No special board initialisation will take place.

## 3.4 Initialising the Port

No special port initialisation will take place.

## 3.5 Initialising the IO Device

No special unit initialisation will take place.

## 3.6 IO Device Online Test

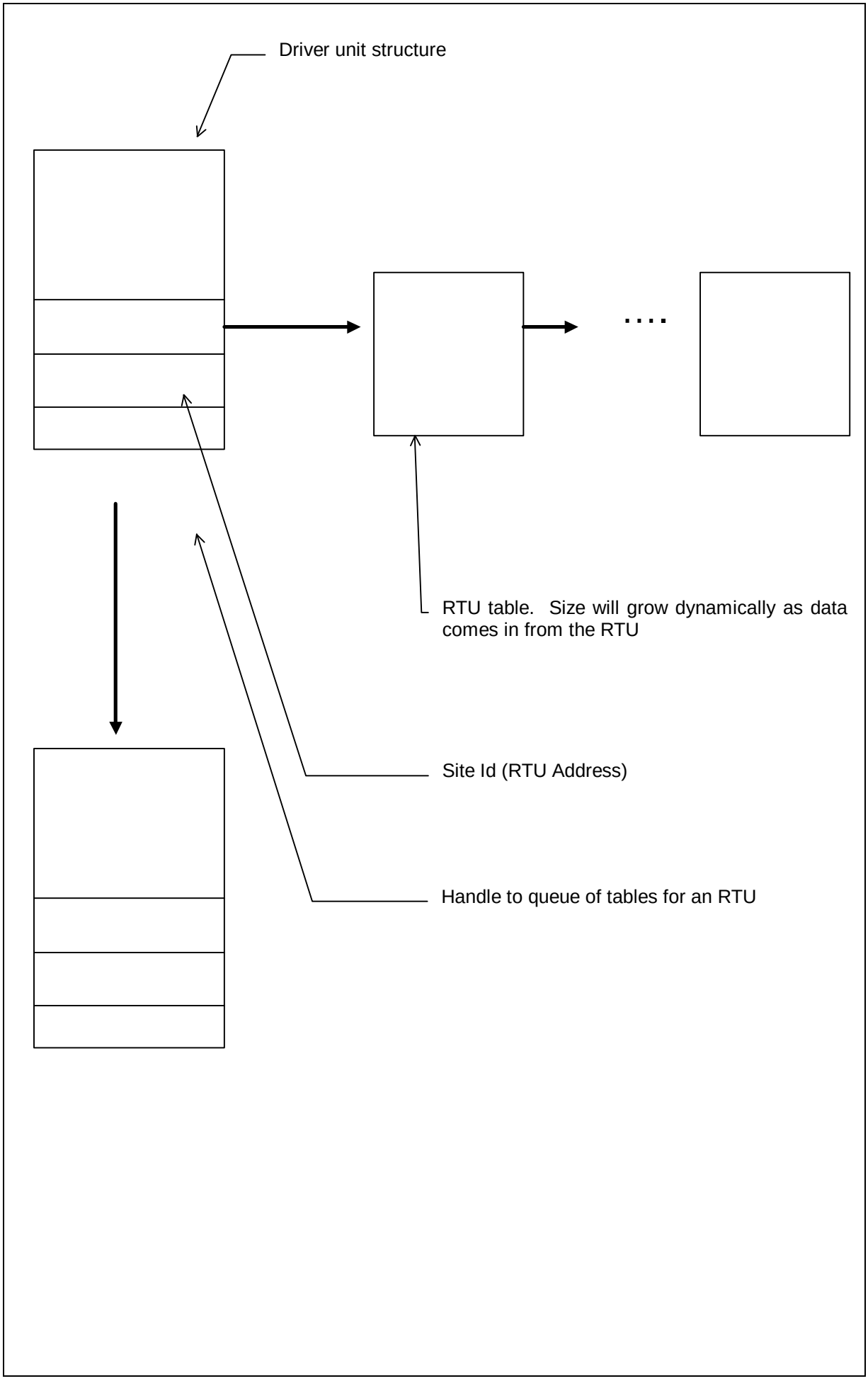
The device will be interrogated for the time.

## 3.7 Internal Data structures for storage of cached data within the driver

Storage structures will be accessed through Citect channel structures, as each Citect channel demarcates a gateway. The maximum size of a table array is 250 rows. An RTU can have 128 tables. A table consists of 1 to 8 columns, each of which can be a different data type. The API can provide information on the number of columns and the data type of each column. The API does not provide any information on how many rows there are in a table (section 6.6.2 in Adobe Acrobat formatted MOSCAD developers guide). In any event a prior knowledge of the number of rows in a table will not be necessary as the driver uses the technique of building up a table in the cache based on the row numbers of incoming data (both from burst event and polled data).

Conceivably, attempts to update the internal cache with poll data may interfere with attempts to update the internal cache with burst data. The use of the **entercritical** function should prevent memory conflicts in this case.

Shown on the following page are the drivers' internal data structures to store cached data. There are 32768 possible RTUs. Each driver unit structure includes a pointer, the pointer points to a queue of structures, each representing a table in the associated RTU.


Driver unit structure
RTU table. Size will grow dynamically as data comes in from the RTU
Site Id (RTU Address)
Handle to queue of tables for an RTU

## 3.8 Cached RTU table format

The format of a cached RTU table is shown below.

| | | Col 0 | Col 1 | Col 2 | Col 3 | Col 4 | Col 5 | Col 6 | Col 7 |
|---|---|---|---|---|---|---|---|---|---|
| | | Col Age | Col Age | Col Age is used for keeping column data in fresh. | | | | | Col Age |
| Row 0 | Row Age | Analog | Real | Discrete | Discrete | Real | Analog | Real | Analog |
| Row 1 | Row Age | Analog | Real | Discrete | Discrete | Real | Analog | Real | Analog |
| Row 2 | Row Age | | | | | | | | |
| Row 3 | Row Age is used for keeping row data in fresh. | | | | | | | | |
| Row 4 | | | | Here is the same structure as the table in RTU. | | | | | |
| Row 5 | | | | | | | | | |
| Row 6 | | | | | | | | | |
| Row 7 | | | | | | | | | |
| Row 8 | | | | | | | | | |
| Row 9 | | | | | | | | | |
| Row 10 | | | | | | | | | |
| Row 11 | Row Age | | | | | | | | |
| Row 12 | Row Age | | | | | | | | |
| | ? ? ? | ? ? ? ? ? ? | | | | | | | |
| | ? ? | ? ? ? | | | | | | | |

## 3.9 Request Queue Structures

The driver will maintain 2 request queues (note these are not the queues of table data). They are:
1) A Citect dcb request queue of inbound requests.
2) A Citect dcb request queue of outbound requests.

## 3.10 State Flow Description

### Summary

The following discussion describes Citect INI parameters. Some of theses parameters are available in both the scope of the entire driver and in the scope of a particular unit(s). These parameters are named informally here to generically refer to both the driver scope version and the unit scope version. To identify the correct parameter name for the scope required, refer to Section 5.9.2.

All read data is cached by the driver. The driver will maintain a list of recently polled data, and will schedule that data again for polling if Citect is still “looking” at the data.
When Citect makes a request for an area of memory that has not been created in the driver (by incoming data due to poll request or burst data), then a request for all the data in those rows will be made.

During the drivers regular scheduled polling of those RTUs configured to be polled, then the driver will only poll for data that has changed state i.e. the driver will request a number of rows from a particular table of a particular RTU with the qualification that the RTU is only to return the data that has changed state from the last poll, this is termed a COS poll.

It should be noted that this driver design diverges from the recommended implementation. The recommended strategy (described in section 7.2.1 of the MOSCAD developers guide) is to poll all the communication tables at startup and then cyclically poll the entire database thereafter to keep all data within some freshness time threshold. As this strategy could have the result of slow data update times for large databases, this driver will use an update strategy whereby RTU table "areas of interest" to the IOServer will be polled while the IOServer is interested in them, but removed from the polling schedule when the IOServer is no longer interested in them (e.g. the user has changed to a different page). This strategy does have the drawback, that there may be some delay on initially updating a page when a new page is chosen by the user. Thereafter though, the update times should be better than those resulting from the other strategy.

It is recommended that the driver poll all RTUs and gateways on a regular basis, regardless of whether they are being currently used by the SCADA system, just to check that they are online. The driver will implement this and return SEVERITY_WARNING level errors where a unit that is not currently being requested goes offline.

### 3.10.1 Cache update strategy

For optimum system performance the driver will rely on receiving change of state burst data from RTU as the primary method to update the cached data structure for an RTU. This will require MOSCAD system configuration from the user, refer to Section 3.5.3.2. The driver will always poll an RTU, the user is to configure the polling rate for the device to reflect the polling requirements. If the device will be ”bursting” unsolicited responses, then the device should only be polled at a very low frequency (just to ensure data integrity), as the bulk of communication will be accomplished through unsolicited response. If the device will not be ”bursting” unsolicited responses, then the device should be polled at a higher frequency, as this will be the only method of data being updated in Citect.

Primary Data Acquisition through Burst Data Change of State (COS) Information
Through RTU Application Configuration (Ladder Logic), the RTU is capable of flagging data that has changed state, and transmitting this new data, without the need for Citect to request it. In this manner, Citect can receive these unsolicted responses, and remain up-todate with the device. A point to note here is that Analog values only have their COS flag set when the value deviation exceeds a threshold, this implies that very slow changes in analog values ( or “creeping” values), will not be updated. The safest method is to also conduct infrequent “integrity polls” to ensure the data integrity.

All Data Acquisition through Polling of RTUs
The cache update technique for this secondary strategy is as follows:

Each row of each table structure (queue item) in the cache will have a "data in" time stamp, recording when data last came in for that row (data will be requested in units of whole rows, and event data is received in units of whole rows).

All data in cache will be updated after a integrity poll or COS poll.

### 3.10.2 Checking Cache freshness

A separate thread will continually scan through cache on a unit-by-unit basis, and generate Read requests in order to refresh the areas of cache that are “stale”.
This thread is also responsible for routinely issuing unit checks on a unit-by-unit basis to monitor the status of all units.

### 3.10.3 Handling Timeouts

When a unit is marked offline for failing to respond for a set amount of retrys, the driver will no longer automatically interrogate the device to update the cache. The unit will only be marked back online when the driver receives a response to the Time Read request initiated by either a Init Unit request from Citect or a Unit Check request from the driver.
This routine will also check for the theoretical case where a DCB has been left on the DCB Out queue and has expired, but the corresponding Custom Request has not expired.

## 3.11 Message Structure

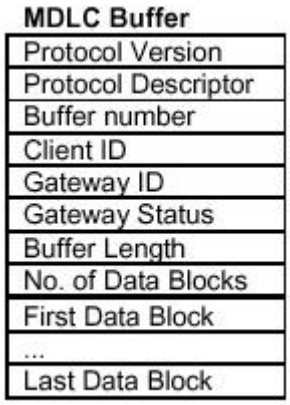
The following is a brief summary of how the following MDLC message will be used in the driver.

The communication between the MDLC Gateway and the SCADA driver is carried out by the Gateway API routines that send/receive MDLC Buffers that are transferred via the TCP/IP protocol, from the SCADA central (client)to the MDLC Gateway (server) and vice versa. Each one of the MDLC Buffer consists of a Buffer and Data Blocks. The Data Blocks have a Data Buffer Header in fixed format and data in various formats.

#### 4.11.1 MDLC Buffer Format

##### 3.11.1.1 MDLC Buffer Header

The MDLC Buffer Header consists of the following information:
1) ***Protocol Version*** (2 Bytes) ????the buffer protocol identifier, must be "1" for MDLC.
2) ***Protocol Descriptor*** (2 Bytes) ????the buffer protocol descriptor (Not Applicable for MDLC).
3) ***Buffer Number*** (4 Bytes) ????the serial number of the buffer, this field is used to identify reply messages.
4) ***Client ID*** (2 Bytes) ????the SCADA Client identifier. This identifier is received during the creation of the channel between the Gateway (server) and the SCADA (client).
5) ***Gateway ID*** (2 Bytes) ????the MDLC Gateway identifier. This identifier is created during the Gateway configuration and is received during the creation of the channel between the Gateway (server) and the SCADA (client).
6) ***Gateway Status*** (4 Bytes) ????the MDLC Gateway status.
7) ***Buffer Length*** (2 Bytes) ????the total buffer length (includes header).
8) ***Number of Blocks in Buffer*** (2 Bytes) ????the number of data blocks that follow in the buffer.



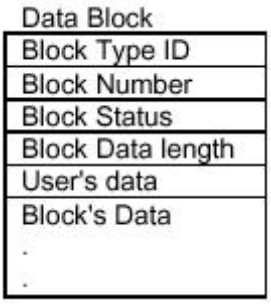

#### 3.11.1.2 Data Blocks Structure

The Data Blocks contain the data to be transferred on the LAN by means of the MDLC Buffers.

Each Data Block is a closed entity that precisely defines the data/request. The Data Blocks consist of a Data Block header in fixed format and a Data Block body in various formats.

The Data Block header contains the definition of the data that is provided in the data part of the Block. It consists of the following:

1) ***Block Type Identifier*** (4 bytes) ????the type of data block (Messages and Operation codes). Definition of all the messages types is given in Chapter 5 of this manual.
2) ***Block Number*** (4 bytes) ????the data block number, this field is used to identify reply messages (assign 0 for the first block, 1 for the second etc.) .
3) ***Block Status*** (4 bytes) - return Code for the block's status. The status can include a return code.
4) ***Block Data Length*** (2 bytes) ????the length of the data part in the Data Block (excluding the block header).
5) ***User's data*** (2 bytes) ????saved for the user's information.




The data in the Data Block varies according to the buffer application. The Block Type Identifier defines the format of the bytes in the Block's Data segment. There are two main data formats:

1) ***Application Data Format*** ????this format is generally used for SCADA Application data that is sent/received to/from the MDLC RTUs. It includes the following:

Application Data Block

| Block Type ID |
|---|
| Block Number |
| Block Status |
| Data length |
| User's data |
| Coordinates |
| Application Data<br>.<br>. |

?? ***Coordinates*** ????the coordinates define a group of destination points for the MDLC communication to access.

?*MDLC Coordinates* : the MDLC coordinates definition includes all the information needed to access a specific set of elements in a MOSCAD RTU.

The maximum number of elements in the MDLC coordinates descriptor is a specific table in a MOSCAD RTU.

The minimum number of elements in the MDLC Coordinates descriptor is a specific element in a MOSCAD RTU.

The MDLC coordinates descriptor includes the following:

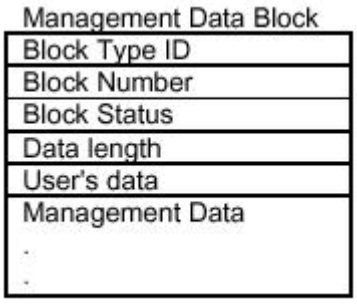
?? *Site ID* (2 Bytes) : the MDLC Site ID of the target RTU (1-65534) 65534 is assigned for the MDLC site_id of the simulator.
?? *Table Number* (1 Byte) : the number of the target table (0-127).
?? *Row Number* (1 Byte) : the number of the starting row (0-249).
?? *Number of Rows* (1 Byte) : the number of consecutive rows starting from Row Number
?? *Column Number* (1 Byte): the number of the first column (0-7).
?? *Number of Columns* (1 Byte): the number of consecutive columns starting from the Column Number (1-8).
?? *Spare*(1 Byte): not applicable, for padding purpose only.
?? *Columns Types* (4 Bytes): Columns Types descriptor (not applicable for MDLC).
?? *User Coordinates* (4 Bytes): saved for the user's SCADA data-base pointer.

MDLC Coordinates

| Site ID |
|---|
| Table Number (Z) |
| Row Number (Y) |
| Number of Rows |
| Column Number (X) |
| Number of Columns |
| Spare |
| Columns Types |
| User Coordinates |

?? ***Application Data*** : the Application Data is the raw data that is transferred between the Client and the MDLC Gateway (Command, Setpoint, Spontaneous data etc.).
A few requests are not include application data (Request for Polling, Get MDLC Gateway time etc.).

2)***Management Data Format*** ????this format is used for the Gateway management protocol (not for the SCADA application protocol) between the Gateway (client) and SCADA (server). It includes the following:




?? ***Management Data*** ????the Management Data that is transferred between the Client and the MDLC Gateway.

### 3.11.2 BASIC SCADA SERVICES - DATA BLOCKS & PROTOCOLS

This chapter provides a description of the Data section in the various types of the Basic SCADA Application Data Blocks and their order.

The return Block Type is the original Block type with MSB on.
(Original Request Block Type BinaryOR MDLC_REPLY_MASK).

Data Transfer for Poll Data (DB MAP)

?? Data types of data transfer have the following format:

Analog ????two bytes.
Real ????four bytes.
Discrete ????depends on the context.

?

?? Request for discrete values only:

Every discrete value is represented by one bit.
Every 8 bits are compressed into one byte.

?

?? Request for discrete values combined with non discrete values:

Every discrete value is represented by one bit.
All the consecutive discrete values are compressed into one byte.

If the request is for several rows and the row starts and ends with a discrete value, then the end of the row is compressed along with the start of the next row into one byte.

Examples for Data Transfer (DB MAP) data format

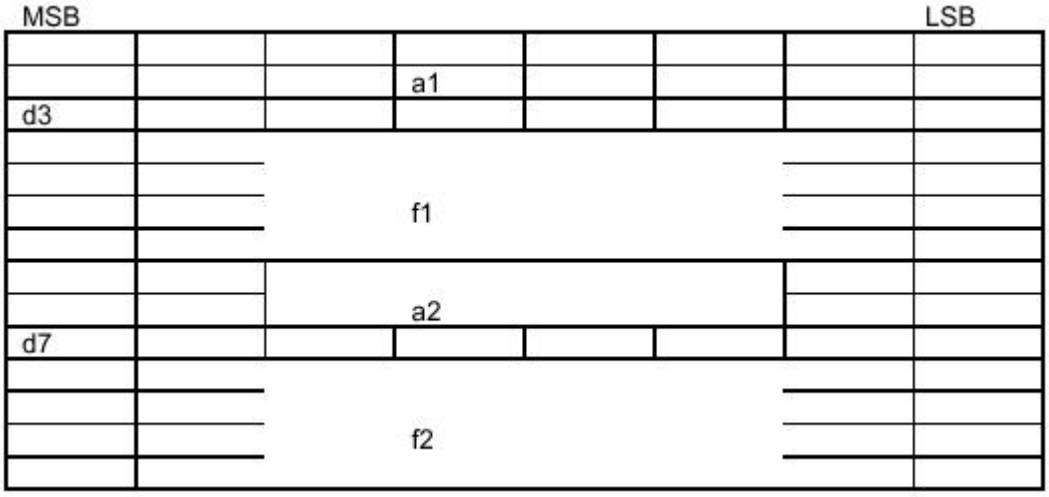
The RTU table description:

| | Column 0 Discrete | Column 1 Discrete | Column 2 Analog | Column 3 Discrete | Column 4 Float |
|---|---|---|---|---|---|
| Row 0 | d1 | d2 | a1 | d3 | f1 |
| Row 1 | d5 | d6 | a2 | d7 | f2 |
| Row 2 | d9 | d10 | a3 | d11 | f3 |
| Row 3 | d13 | d14 | a4 | d15 | f4 |
| Row 4 | d17 | d18 | a5 | d19 | f5 |

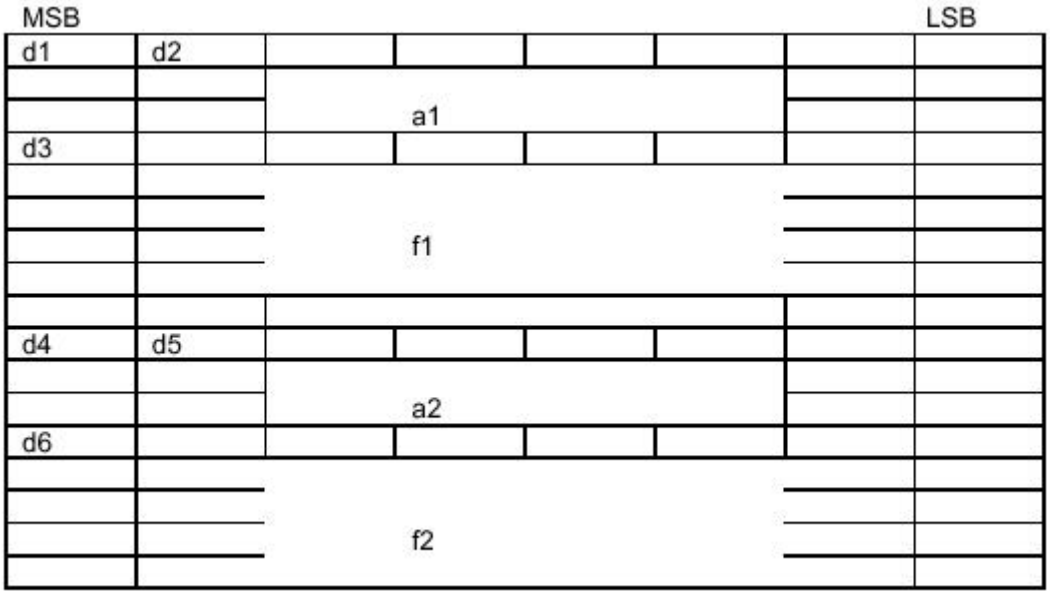
Example 1: Request for Row0 to Row4, Column0 to Column1

| MSB | | | | | | | LSB |
|---|---|---|---|---|---|---|---|
| d1 | d2 | d5 | d6 | d9 | d10 | d13 | d14 |
| d17 | d18 | | | | | | |

Example 2: Request for Row0 to Row1, Column2 to Column4




Example 3: Request for Row0 to Row1, Column0 to Column4




### 3.11.2.1 Poll Data Protocol:

A Poll Data message allows the Client's process Sending request for Polling to a specific MOSCAD RTU, via the MDLC Gateway.

*Client to Gateway* ????Header with Poll Data Code
?? Coordinates

*Gateway to Client* ?????
?? Coordinates
?? Data Returned by DB MAP

| **Client to Gateway** | **Gateway to Client** |
|---|---|
| **POLLING Request** | **Response** |
| **Data Block** | **Data Block** |
| Data Block Type<br>Poll Data Code | Data Block Type<br>Poll Data Code |
| Block Number | Block Number |
| Block Status=0 | Block Status |
| Data Length | Data Length |
| User's data | User's data |
| Coordinates | Coordinates |
| None | Data Returned by DB MAP |

The return Block type is the original Block type with MSB on. (Original Request Block Type BinaryOR MDLC_REPLY_MASK).

### 3.11.2.2 Poll Data with COS (Change Of State) Protocol:

Poll Data with a COS message allows the Client's process sending request for polling with COS to a specific MOSCAD RTU, via the MDLC Gateway.

*Client to Gateway* ????Header with Poll Data with COS Code
?? Coordinates

*Gateway to Client* ?????
?? Coordinates
?? Data Returned by DB MAP

| **Client to Gateway** | **Gateway to Client** |
|---|---|
| **POLLING COS Request** | **Response** |
| **Data Block** | **Data Block** |
| Data Block Type<br>Poll COS Data Code | Data Block Type<br>Poll COS Data Code |
| Block Number | Block Number |
| Block Status=0 | Block Status |
| Data Length | Data Length |
| User's data | User's data |
| Coordinates | Coordinates |
| None | Data Returned by DB MAP |

The return Block type is the original Block type with MSB on. (Original Request Block Type BinaryOR MDLC_REPLY_MASK ).
The format of the data returned by DB MAP Data Block in the case of COS is the following:

DB MAP Data Block

| |
|---|
| Row Number |
| A complete Row Data<br>in DB MAP Format<br>.....<br>.... |
| Row Number |
| A complete Row Data<br>in DB MAP Format<br>.....<br>.... |
| .<br>.<br>. |
| 0XFF |

The Site_id and the Table_no(Z) are specified in the Coordinates descriptor, while the Row_no(Y) (1 Byte) is specified in the Application Data Block which include the Row Data in DB_MAP format.

If there is no Change Of State to report, then only 0XFF is returned as data.

#### 3.11.2.3 Burst Data Transfer Protocol:

Burst Data Transfer message allows sending contention data from the MDLC Gateway to the Client's process.
*Client to Gateway* ????None
*Gateway to Client* ?????
?? Coordinates
?? Data Received by Burst Contention

| Client to Gateway | Gateway to Client |
|---|---|
| Data Block | Data Block |

| Data Block Type |
|---|
| Burst Data Code |
| Block Number |
| Block Status |
| Data Length |
| User's data |
| Coordinates |
| Data Received by Burst Contention |

The Site_id is specified in the Coordinates descriptor.

The format of the data returned by Burst Data Block is the following:
**Z** - Table number (1 Byte)
**Y** - Row number (1 Byte)
Data - Row's Data in Burst Format
The row data in a BURST Data Block is represented as in DB MAP Format except:
?? Burst of discrete values: Every discrete value is represented by one byte.

#### 3.11.2.4 RTU Event Data Transfer Protocol:

RTU Event Data Transfer message allows sending RTU events from the MDLC Gateway to the Client's process.
*Client to Gateway* ????None
*Gateway to Client* ?????

| Burst Data Block |
|---|
| Table Number-BURST 1 |
| Row Number |
| A complete Row Data in Burst Format ..... .... |
| . |
| . |
| . |
| Table Number -BURST n |
| Row Number |
| A complete Row Data in Burst Format ..... |

?? Coordinates
?? Hours.
?? Minutes.
?? Seconds.
?? Event Code.
?? Event Index.

| Client to Gateway | Gateway to Client |
|---|---|
| | Event Code |
| Data Block | Data Block |

| Block Number |
|---|
| Block Status |
| Data Length |
| User's data |
| Coordinates (Site ID) |
| Hour |
| Minute |
| Second |
| Event Code |
| Event Index |

The Site_id is specified in the Coordinates descriptor.

#### 3.11.2.5 Send Data Block (for Send Data (DB MAP) Applications)

Send Data message allows sending commands as far as downloading parameters to specific MOSCAD RTU via the MDLC Gateway.
*Client to Gateway* ????Header with Send Data Code
?? Coordinates
?? Data Returned by DB MAP

*Gateway to Client*
?? Header with Return Status

Send Data Protocol:

| Client to Gateway<br>Send Data Request<br>Data Block |
|---|
| Data Block Type<br>Send Data Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's data |
| Coordinates |
| Send Data in DB MAP Format |

| Gateway to Client<br>Response<br>Data Block |
|---|
| Data Block Type<br>Send Data Code |
| Block Number |
| Block Status |
| Data Length |
| User's data |
| Coordinates |
| None |

### 3.11.3 GATEWAY MANAGEMENT - DATA BLOCKS & PROTOCOLS

The following is a description of the data section in the various types of management Data Blocks.

#### 3.11.3.1 Connect Channel Data Block

Connect Channel message allows the SCADA Client to establish logical connection with the Gateway to send data, to receive data, and to receive contentions to/from the MDLC Gateway.

*Client to Gateway* ?????

?? Header with Connect Channel Code

*Gateway to Client* ?????

?? Channel ID (Ushort).
?? Client ID (Ushort).
?? Gateway ID (Ushort).
?? Channel Type (Utiny).

Connect Channel Data Block Protocol (Synchronous Operation):

| **Client to Gateway<br>Connect Channel<br>Request<br>Data Block** |
|---|
| Data Block Type<br>Connect Channel Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's data |
| Channel Type |
| Client Name |

| **Gateway to Client<br>Response<br>Data Block** |
|---|
| Data Block Type<br>Connect Channel Code |
| Block Number |
| Block Status |
| Data Length |
| User's data |
| Channel ID |
| Channel Type |
| Client ID |
| Gateway ID |

### 3.11.3.2 Disconnect Channel Data Block

Disconnect Channel message allows the SCADA Client to disconnect logical connection with the Gateway to terminate the Client's open session with the MDLC Gateway.

*Client to Gateway* ?????
?? Header with Disconnect Channel Code

*Gateway to Client* ????Header with returned Status.

Disconnect Channel Protocol (Synchronous Operation):

| **Client to Gateway Disconnect Channel Request Data Block** |
|---|
| Data Block Type Disconnect Channel Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's data |
| None |

| **Gateway to Client Response Data Block** |
|---|
| Data Block Type Disconnect Channel Code |
| Block Number |
| Block Status |
| Data Length |
| User's data |
| None |

### 3.11.3.3 Close Client Data Block

Close Client message allows the SCADA Client to disconnect ALL logical connections with the Gateway to terminate all the Client's open sessions with the MDLC Gateway.

*Client to Gateway* ????Header with Close Client Code
?? Client ID (Ushort).
?? Client Name.

*Gateway to Client* ????Header with returned Status.
Close Client Protocol:

| **Client to Gateway Close Client Request Data Block** |
|---|
| Data Block Type Close Client Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's Data |
| Client ID (2) |
| Client Name |

| **Gateway to Client Response Data Block** |
|---|
| Data Block Type Close Client Code |
| Block Number |
| Block Status |
| Data Length |
| User's Data |
| None. |
| |

### 3.11.3.4 Set Gateway Mode Data Block

The ***Set Gateway Mode*** message allows the SCADA Client to set the Gateway's mode of operation. This message supports the redundant Gateway feature (Primary/Secondary Gateways).
The following modes are available:
Secondary - in which the Gateway responds only to Gateway management messages all other Gateway messages (to/from the MDLC links) are ignored.
Primary - in which the Gateway responds to both the SCADA application messages as well as to Gateway management messages.

*Client to Gateway* ????Header with Set Gateway Mode Code
?? Gateway Mode (Utiny)

*Gateway to Client* ????Header with returned Status.

Set Gateway Mode Protocol (Synchronous Operation):

| **Client to Gateway Set Gateway Mode Request Data Block** |
|---|
| Data Block Type<br>Set Gateway Mode Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's Data |
| Gateway Mode |

| **Gateway to Client Response Data Block** |
|---|
| Data Block Type<br>Set Gateway Mode Code |
| Block Number |
| Block Status |
| Data Length |
| User's Data |
| None |

### 3.11.3.5 Get Gateway Mode Data Block

Get Gateway status message allows the SCADA Client to read the Gateway's mode of operation as well as the Gateway status (Ping-like message). The following modes are available:

Secondary - The Gateway responds only to Gateway management messages, all other Gateway messages (to/from the MDLC links) are ignored.

Primary - The Gateway responds to both the SCADA application messages; as well as to Gateway management messages.

*Client to Gateway* ????Header with Get Gateway Status Code

*Gateway to Client* ????Header with returned Status.
?? Gateway Status and Mode (Ulong)

Get Gateway Status Protocol (Synchronous Operation):

| Client to Gateway<br>Get Gateway Status Request<br>Data Block |
|---|
| Data Block Type<br>Get Gateway Status Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's Data |
| None |

| Gateway to Client<br>Response<br>Data Block |
|---|
| Data Block Type<br>Get Gateway Status Code |
| Block Number |
| Block Status |
| Data Length |
| User's Data |
| Gateway Status and Mode |

#### 3.11.3.6 Get Link Status Data Block

Get Link status message allows the SCADA Client to retrieve the RTU links' type, IDs, and status.

*Client to Gateway* ????Header with Get Link Status Code

*Gateway to Client* ????Header with returned Status.

?? Link Type (Ushort)
?? Primary Link ID (Utiny)
?? Secondary Link ID (Utiny)
?? Link Status (Ulong)

Get Link Status Protocol (Synchronous Operation):

| Client to Gateway<br>Get Link Status Request<br>Data Block |
|---|
| Data Block Type<br>Get Link Status Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's Data |
| Coordinates |
| None |

| Gateway to Client<br>Response<br>Data Block |
|---|
| Data Block Type<br>Get Link Status Code |
| Block Number |
| Block Status |
| Data Length |
| User's Data |
| Coordinates |
| Link Type |
| Primary Link ID |
| Secondary Link ID |
| Link Status |

### 3.11.4 Gateway Issues

?? Still no definitive documentation on what errors the MOSCAD gateway is capable of producing, and under what conditions it will report them. As an example – a failure of the gateway to communicate with an RTU while executing a poll command will return the error code 0x81, but wen executing a time read under the same conditions it will return the error code 0x3B.
?? The gateway is very complicated with numerous parameters and exhibits behaviour that is not always understood. The gateway log should be monitored to determine if it is being overdriven by the driver, or if it has its own problems.

?? The gateway will do a health check of all configured RTUs at startup – there is no way to prevent this. If the health check flag is set it will continually recheck all RTUs. This will interfere with the drivers operation and make it very difficult to determine why the driver appears to be malfunctioning. Refer to “Additional information regarding gateway operation”.

### 3.11.5 EXTENDED SCADA SERVICES - DATA BLOCKS & PROTOCOLS

This chapter provides a description of the Data section in the various types of the Extended SCADA Application Data Blocks and their order.

#### 3.11.5.1 Broadcast Data Protocol

The Broadcast application function allows the SCADA to send controls simultaneously to a sub-group of MOSCAD RTUs.

The nature of a Broadcast is that cknowledge is not received from the addressed RTU. Therefore, it should be used only when a simultaneous execution in the addressed RTUs is required.

The Broadcast command may be used to modify the RTU's mode of operation or serve as a trigger for a specific operation.

The target of the Broadcast message is a specific table in the RTU's data base, named Qualification table (always exists in the data base). It is the first table of the user-defined tables.
The user defines his variables using meaningful symbolic names.

***NOTE:*** *The same variables should be defined in this table for all RTUs in the system.*

The data type of the variables in this table is **Integer Parameters** (16-bit integers defined with their start time (initialization) value).

If Q and M are two variables defined in the Qualification table, then the format of the Broadcast sent to the RTU is:

“If Q=a then set M=b”
where a and b are integer constants.

In response to the Broadcast command, in all RTUs where Q=a, the variable M is assigned the value b.

By setting the qualifier Q in the RTUs to different values, any number of RTU groups may be formed. The value of Q in each RTU may be changed to move the RTU to a different group.

Example
Refer to the following parameters defined in the Qualifier table:
· *Color* ????sorts the RTUs according to their color.
· *Mode* ????controls the mode of operation of the RTU.

The following Broadcast may be sent:
If Color=red then set Mode=idle,
where red and idle are constants defined in the RTU Constants tables.

This command sets the mode of operation of all the RTUs that belong to the “Red group” to idle.
Any number of variables may be defined in the Qualification table to form any number of grouping criteria and controls and triggers.

*Client to Server* ????Header with Broadcast Code
?? Qualification row.
?? Qualification data.
?? Qualification comparison type.
?? Set row.
?? Set data.

*Server to Client* ????Header with returned Status

Broadcast Protocol:

| **Client to Gateway Broadcast Data Block** |
|---|
| Data Block Type Broadcast Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's Data |
| Coordinates |
| Qualification Row |
| Qualification Data |
| Qualification Comparison |
| Set Row |
| Set Data |

| **Gateway to Client Response Data Block** |
|---|
| Data Block Type Broadcast Code |
| Block Number |
| Block Status |
| Data Length |
| User's Data |
| Coordinates |
| None |

### 3.11.5.2 Get RTU’s Date & Time Data Protocol

The Get RTU’s Date & Time function allows the SCADA to upload any RTU’s date & time to the SCADA central.

*Client to Server* ????Header with Get RTU Time Code

*Server to Client* ????Header with returned Status
?? Year.
?? Month.
?? Day.
?? Day of Week.
?? Hours.

?? Minutes.
?? Seconds.

Get RTU Time & Date Protocol:

| **Client to Gateway Get Time Data Block** |
|---|
| Data Block Type Get Time Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's Data |
| Coordinates |
| None |

| **Gateway to Client Response Data Block** |
|---|
| Data Block Type Get Time Code |
| Block Number |
| Block Status |
| Data Length |
| User's Data |
| Coordinates |
| Year |
| Month |
| Day |
| Day of Week |
| Hour |
| Minute |
| Second |

### 3.11.5.3 Set RTU's Date & Time Data Protocol

The Set RTU's Date & Time function allows the SCADA to download the SCADA central date & time to any RTU in the system.

*Client to Server* ????Header with Set RTU Time Code
?? Year.
?? Month.
?? Day.
?? Day of Week.
?? Minutes.
?? Seconds.
?? Hours.

*Server to Client* ????Header with returned Status

| Client to Gateway<br>Set Time Code<br>Data Block |
|---|
| Data Block Type<br>Set Time Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's Data |
| Coordinates |
| Year |
| Month |
| Day |
| Day of Week |
| Hour |
| Minute |
| Second |

| Gateway to Client<br>Response<br>Data Block |
|---|
| Data Block Type<br>Set Time Code |
| Block Number |
| Block Status |
| Data Length |
| User's Data |
| Coordinates |
| None |

### 3.11.5.4 Sync RTUs Time Data Protocol

The Sync RTUs Time function allows the SCADA to synchronize all the RTUs connected to the MDLC Gateway with the current Gateway time.

*Client to Server* - Header with Sync RTU Time Code
?? Coordinates (Site_ID = 0 for all RTUs)

*Server to Client* - Header with Returned Status
?? Coordinates (Site_ID = 0 for all RTUs)

| Client to Gateway<br>Sync RTUs Code<br>Data Block |
|---|
| Data Block Type<br>Sync RTUs Code |
| Block Number |
| Block Status=0 |
| Data Length |
| User's Data |
| Coordinates |
| None. |

| Gateway to Client<br>Response<br>Data Block |
|---|
| Data Block Type<br>Sync RTUs Code |
| Block Number |
| Block Status |
| Data Length |
| User's Data |
| Coordinates |
| None. |

# 4 User Interface

## 4.1 Introduction

This section defines how the user will see the driver. This relates directly to how the Citect forms need to be filled out and any special INI options. For the kernel, the debug trace messages and the Stats.Special counters are documented.

## 4.2 Driver Name

MOSCAD

## 4.3 Boards Form

| Field | Default | Allowable values |
|---|---|---|
| Board Name | This field is user defined, and is not used by the driver. | |
| Board Type | TCPIP | |
| Address | 0 | |
| I/O Port | BLANK | |
| Interrupt | BLANK | |
| Special Opt | BLANK | |
| Comment | This field is user defined and is not used by the driver. | |

## 4.4 Ports Form

(A port maps to a gateway.)

| Field | Default | Allowable values |
|---|---|---|
| Port Name | This field is user defined and is not used by the driver. | |
| Port number | Must be unique, but is not used by the driver | |
| Board name | Refers to the board previously defined in 'boards' form. | |
| Baud rate | BLANK | |
| Data bits | BLANK | |
| Stop bits | BLANK | |
| Parity | BLANK | |
| Special Opt | PLC's IP address* | |
| Comment | This field is user defined and is not used by the driver. | |

* The IP address must use the following format: **-Ia**

Where: **a** is the destination IP address in standard Internet dot format

Example: **–I280.9.21.60**

There may or may not be a hot backup gateway. If there is a hot backup defined then the user must follow the instructions detailed in section 2.5.3.2 Gateway Mode of Operation Configuration

## 4.5 IO Devices Form

(An IO Device will map to an RTU.)

| Field | Default | Allowable values |
|---|---|---|
| Name | This field is user defined, and is not used by the driver. | |
| Number | Must be unique, but is not used by the driver. | |
| Address | # NameOfDevice | |
| Protocol | MOSCAD | |
| Port name | Refers to the port previously defined in 'ports' form. | |
| Comment | This field is user defined and is not used by the driver. | |

**Address : # NameOfDevice**

where:
# is the RTU address between 1 and 32768
*NameOfDevice* is an optional field that associates this unit with the parameters defined in the Citect INI file defined by the section '[*NameOfDevice*]'.
If this optional field is not utilised, then this unit will just use the values defined by the global INI parameters defined by the [MOSCAD] section.
Note that for units that require identical configuration of INI parameters, they can all reference the same *NameOfDevice,* and hence be associated with the same INI section.

The critical RTU must be the first device in the IO Devices form

## 4.6 Pulldown lists Help

The following entries should be included in the Citect HELP.DBF spec file.

| TYPE | DATA | FILTER |
|---|---|---|
| PROTOCOL | **MOSCAD** | |
| BOARD | **MOSCAD** | |

## 4.7 IO Device Variable Types

### 4.7.1 Formats and types

| IO Device Type | Citect data format | Citect data types | Description/Special Usage/Limitations/ Valid Ranges |
|---|---|---|---|
| Bit | Bz.y.x | DIGITAL | Read / Write |
| Compressed Bit | CBz.y.x | DIGITAL | Read Only |
| Analog | Az.y.x<br>Az.y.x.w | INTEGER<br>DIGITAL | Read / Write<br>Read Only – Single bit of analog |
| Float | Fz.y.x | REAL | Read / Write |
| TimeStamp for data element<br>(time in seconds since 01/01/1970 in UCT format) | TSz.y.x | LONG | Read Only<br>The z.y.x address of a timestamp, when constructing a Citect tag, will be the z.y.x address of the data element that the timestamp corresponds to.<br>The Timestamp will physically reside in the first two columns for row timestamped data or in the following two columns for individually timestamped data |
| RTU Time<br>(time in seconds since 01/01/1970 in UCT format) | RTUTime | LONG | Used to read RTU Date / time<br>Read Operations on this tag will read the value from diver cache.<br>Write Operations on this tag will update the value held in driver cache (this prevents continuous device requests from a screen tag)<br>Use Cicode functions for displaying time/date. |
| Synchronise RTU time with Citect | SynchRTUTime | DIGITAL | Write Only<br>Driver will always return zero if user does try to display tag |
| Force Poll of RTU | ForcePoll | DIGITAL | Write Only |
| Force COS Poll of RTU | ForceCOSPoll | DIGITAL | Write Only |
| RTU Status | RTUStatus.[<<br>STALE, EPOLL,<br>IPOLL, POLL,<br>AV, INUSE,<br>CRIT>] | DIGITAL<br>DIGITAL | Read Only – status of RTU tag is assigned to |
| Broadcast | BCast *a.b.c* | INTEGER | Write Only<br>*a* = TargetRow |

| | | | |
|---|---|---|---|
| | | | *b* = TestData<br>*c* = TestRow<br>Argument = TargetData<br>If TestRow of Table 0 of an RTU holds the value TestData, then TargetRow of Table 0 of that RTU will be loaded with TargetData<br>*(Note that this format is only applicable for 'virtual units' – memory or disk PLCs. Refer to note below on virtual units)* |
| Cache Age | CAGEz.y.x | REAL | Cache Age |
| Tag Status | STATUSx.y.x | REAL | Tag Status |

Where:

| | | |
|---|---|---|
| *z* | the table number in the RTU data base | 0..127 |
| *y* | the row number in the table | 0..249 |
| *x* | the column number in the table | 0..7 |
| *w* | The selected bit of an analog | 0..15 |

*Virtual unit* A virtual unit is a unit that is defined in the Devices form as a memory or disk PLC, and has special tags assigned to it in order to interact with the driver. In this case, a virtual unit is created for checking the status of real units, so that the status tag will not be #COM'd when the unit is offline. A virtual unit is also created for the broadcast operation, as this operation effects multiple units, and hence a broadcast tag is not related to just one unit. There are INI parameters to distinguish the unit address as virtual units, refer to Section 5.9.2 for details.

### 4.7.2 Event data types (alarm and trend)

This section is concerned with the processing of time stamp data. A data element in an RTU table is timestamped by the RTU application configuration placing time stamp data in the two columns following the data element. The first column is of type TS1 and the second column is of type TS2. For further information refer to Section 3.5.3.2. When a data element and its time stamp are received by the driver, the data and the time stamp are placed in the cache. This means that the data and the time stamp are available for normal tag read operations.
In addition to this, the data and its associated time stamp, as well as other relevant information, is placed onto an event queue. This queuing system, and the interrogation of the queue through Cicode, was developed for the KINGFISHER driver.

Each event queue item contains the following information:

| |
|---|
| TIMESTAMP in Seconds Since 01/01/70 (In Universal Coordinate time) – provides Date and Time |
| MILLISECONDS Since Midnight – provides Millisecond Accuracy for digital alarms |
| VALUE |
| VALUE TYPE (2=Bit, 4=Analog or 5=Float) |
| TAG NAME |

Refer to 5.7.3 for further details.

The event data received may, or may not, contain timestamps. Either way the value contained in the event data is used to update the driver's cache. If the event data is time-stamped, then the driver will check to see if the event data is required for Time Stamped Alarms or for Trends. If it is, then it will pass the event data onto the driver's event que, which will then in turn be interogated by dedicated Cicode, that will process the event data and update the associated Alarm or Trend.

The project must initiate the dedicated Cicode if event driven alarms or trends are to be used. This is accomplished by defining a *Parameters* form (from the *System* menu) as follows:

Section Name: Code
Name: Startup
Value: Tasknew("RTUEventQueProcessing",0)

To simplify the matching process of determining the trend tag associated with an incoming time stamped unsolicited response, the following naming convention is used. For Periodic Trends, determine the tagname that corresponds to the I/O point from which the event originated, and suffix the tagname with "T_". For instance, if B1.0.0 generates an event, and has a tag PUMP_20_RUNNING defined for it, then the corresponding trend tag should be named T_PUMP_20_RUNNING. By using the tag PUMP_20_RUNNING as the trend tags expression, the trend will continue to trend the value held in cache in between events. Note that the event will be inserted into the trend at the time in which the event occurred, not the time at which the event was received. The dedicated Cicode will then redraw the trend (with the new value) from the time of the event up until the present time. Similarly, for a Periodic Event trend, the trend tagname should be suffixed with "P_". For example, P_PUMP_20_RUNNING.

In the future, Event trends will be supported by the "E_" suffix, as in E_PUMP_20_RUNNING.

*Trends for analog received events are not currently supported* , but will be implemented in the same manner as that described for digital received events, once they are supported.

Only use standard templates when constructing trend displays to be used by event driven trends. If a "zoomtrend" is to be used, then set the [DNP] parameter *ZoomTrendInUse=1.*

It is still possible to use normal Citect trends. By using a trend name that does not follow the above naming convention(eg simply using the tagname as the trend tagname), then the trend will be of the value held in driver cache and will be relative to Citect time. This implies that changes in value will relate to the time at which the data was received, rather than when the event occurred.

Note that the trending system does not support the importing of data in millisecond resolution, hence the main use of trending time stamped data is in a system where there is unacceptable, or variable, delay between when the RTU scans its I/O and when Citect receives and timestamps the data. If this is not an issue, then normal trending can be used, with Citect trending the data held in the driver cache. Note that the driver cache will be updated in between polls if an unsolicited response is received.

For each time stamped digital alarm configured in the RTU to be "burst" to Citect, two tags are to be defined in the following format. Determine the tagname that corresponds to the I/O point from which the alarm originated, and suffix the tagname with "A_". For instance, if B1.0.1 generates an event, and has a tag PUMP_20_TRIPPED defined for it, then the corresponding Time Stamped Alarm tag should be named A_PUMP_20_TRIPPED. The *variable* tag for the alarm should be suffixed by "V_", as in V_PUMP_20_TRIPPED, and the *timer* tag should be suffixed by "S_", as in S_PUMP_20_TRIPPED. Both the "V_" and "S_" tags should be virtual tags defined for a memory PLC, with the "V_" tag having a D*n* address and of type Digital, and the "S_" tag having a L*n* address and of type Long.
Cicode will extract events from the driver's event queue, and if the data is digital, write the time stamp to the "S" tag, and then write the value to the 'V" tag (if they have been configured). This will trigger the Time Stamped Alarm tag, which will then display with the state and time of occurrence as timestamped in the RTU.

As the Alarm system polls tags for changes in value, but events are received compressed in time, then any consecutive events **for the same point** must be delayed by an interval equal to or greater than the polling time used by the alarm system. If this is not assured, then there is a risk of alarms being missed as the events will be fed to the alarm server quicker than it is polling for changes. The [ALARM] parameter *ScanTime* allows the user to control the actual trend system polling time. The driver reads this value, and enforces its own minimum of 1 second, so that it can enforce this delay between consecutive events of the same point. The "Citect Info" function can be used to determine the "Maximum time between code execution (cycles)" for Time Stamped Alarms – CitectInfo("Stats","High Res. Al",1). The *ScanTime* parameter must be adjusted so as to ensure that the actual alarm scan time does not exceed the expected alarm scan time.

Note also that the timestamp can be viewed in one of two forms, Date/Time or Time/Millisecond. The form to be used is defined by the [ALARM] parameter *HresType*. Setting *HresType=4* defines Date/Time format, and setting *HresType=7* defines Time/Millisecond format. A format must be selected in order for device timestamped alarms to work, so an error will be generated if it is not defined in the CITECT.INI file. The [ALARM] parameter *HighResOff* should be set to 1 (*HighResOff=1*) to force millisecond accuracy when alarms are turned off, if Time/Millisecond format is being used. An appropriate alarm category must also be defined to correctly utilise the selected format.

**Refer to the sample project for examples of event driven Trend tags and event driven Time Stamped Alarm tags.**

### 4.7.3 Cicode functions to access timestamped event data

GetRTULogStatus()

| | |
|---|---|
| Purpose: | To indicate whether there are records available for extraction from the internal driver event cache. |
| Return Value: | A value representing the number of records remaining in the driver cache (LONG). |
| Parameters: | NONE |

GetRTUNextEvent()

| | |
|---|---|
| Purpose: | To remove the event record from the front of the internal driver event cache, and make it available for field data extraction using GetRTUEventField. |
| Return Value: | A handle to the first event OR -1 if no events in internal queue (LONG) |
| Parameters: | NONE |

GetRTUEventField(hEvent, fieldNumber)

| | | | |
|---|---|---|---|
| Purpose: | To obain a field from the current extracted event record. | | |
| Return Value: | Selected event record field as a string | | (STRING) |
| Parameters: | hEvent | - event handle | (LONG) Range 1-65,535 |
| | fieldNumber | - field number | (LONG) Range 1-5 |

| Field Number | Field Name | Field Description | Cicode Numerical Type. (Cicode to convert field from STRING to the listed type) |
|---|---|---|---|
| 1 | TIMESTAMP | Time in seconds since 1970, in Universal Coordinate Time | INT |
| 2 | MILLESECONDS | Milliseconds since midnight – additional accuracy for optional use in Time Stamped Digital Alarms. | INT |
| 3 | VALUE | Value of event | Bit INT<br>Analog INT<br>Float REAL |
| 4 | VALUE TYPE | Used to determine data type of value (2=Bit, 4=Analog or 5=Float) | INT |
| 5 | TAG NAME | Tag Name | STRING |

EmptyEventQue()

| | |
|---|---|
| Purpose: | To empty the internal driver event queue. |
| Return Value: | Always returns 0. |

### 4.7.4 MOSCAD.dbf Entries

Notes on driver dbf.

This driver is unusual in that the user can configure the structure of the RTU communication tables (Citect data types). Each table communication tables can have a number of columns between 1 and 8 and a number of rows between 1 and 250. In addition, the data type make up of the rows can also vary from table to table (eg int, int, float, bit or float, int, bit , bit, bit, bit). This variability makes it difficult for the Citect compiler to block requests correctly. It is possible for the driver to block along columns, so this method will be permitted by the driver where possible.

Blocking along columns is used for types Analog (A), Float (F) and TimeStamp(TS) because these types have a bit width greater than 8 bits, which is the minimum size that the IO server will block correctly (1 bit blocking does not work properly). This blocking approach relies on continuity of address space (assured in a table column) and that the IOServer will not request an address larger than the largest address (row) the user uses as a tag for a particular UnitType (column) i.e. any "incorrect address" errors will be a result of the user adding an incorrect tag rather than as a result of the IO server's optimisation.

The Bit type (B) cannot be blocked at all, because no blocking along rows is possible (due to type variability) and no blocking along columns is possible (as it cannot be guaranteed that there will be multiples of 8 rows).

From this scheme then:
Bits 0..7 of the UnitType field are reserved for the base type (A, F… etc)
Bits 8..15 are reserved for the table number where required
Bits 16..23 are reserved for the column number where required

The driver has access to information of the makeup the columns and rows from the API, and so can optimise the requests from Citect.

Note that since the driver will block along columns rather than rows (i.e. against the natural order the API uses), the writing of multiple data items e.g. strings will result in one Citect write request being split into multiple API requests to write individual data items.

| TEMPLATE | UNIT TYPE | RAW TYPE | BIT WIDTH | LOW | HIGH | COMMENT |
|---|---|---|---|---|---|---|
| A%<8(0,1,128).%U(0,0,249).%<16(0,0,7)[.%u(0,0,15)] | 1 | 1 | 16 | 0 | 0 | Analogues |
| F%<8(0,1,128).%U(0,0,249).%<16(0,0,7) | 2 | 2 | 32 | 0 | 0 | Floats |
| TS%<8(0,1,128).%U(0,0,249).%<16(0,0,7) | 3 | 4 | 32 | 0 | 0 | TimeStamp |
| CB%<8(0,1,128).%U(0,0,249).%u(0,0,7) | 4 | 8 | 8 | 0 | 0 | Compressed Bits |
| B%<8(0,1,128).%<U(0,0,249)%*8.%<16(0,0,7) | 5 | 0 | 1 | 0 | 0 | Bits |
| RTUTime | 6 | 4 | 32 | 0 | 0 | RTU time in seconds |
| RTUStatus[.%u<STALE,EPOLL,IPOLL,POLL,AV,INSUE,CRIT>] | 7 | 1 | 16 | 1 | 32768 | Online status of RTU |
| SynchRTUTime | 8 | 0 | 1 | 0 | 0 | Synchronise RTU time to Citect time |
| BCast%<8(0,0,249).%U(0,0,65535).%<16(0,0,249) | 9 | 1 | 16 | 0 | 0 | Broadcast qualified control message (BCASTTargetRow.TestData.TestRow) |
| ForcePoll | 10 | 0 | 1 | 0 | 0 | Force a poll (non-COS) of unit |
| ForceCOSPoll | 11 | 0 | 1 | 0 | 0 | Force a COS (Change Of State) poll of unit |
| CAGE%<8(0,1,128).%U(0,0,249).%16(0,0,7) | 12 | 2 | 32 | 0 | 0 | Cache age |
| STATUS%<8(0,1,128).%U(0,0,249).%<16(0,0,7) | 13 | 2 | 32 | 0 | 0 | Tag status |

## 4.8 PROTDIR.DBF

MaxBits calculated as

Max size of Moscad receive buffer = 20480 * 8 = 163840 bits

Max size of Moscad table row = 8 columns * sizeof(float) = 8 * 32 = 256 bits.

Max no of Max Size rows that can be read in one request from Moscad = 163840 / 256 = 640 rows.

Max size of Citect request of floats = 2048 / 32 = 64 items (means driver requests 64 rows because of blocking along columns).

Therefore MaxBits is maximum possible for Citect ie 2048 bits.

| TAG | FILE | BIT_BLOCK | MAX_LENGTH | OPTIONS |
|---|---|---|---|---|
| MOSCAD | MOSCAD | 2048 | 2048 | 0x10cb[*1] |

[*1] - Supports Digital, Integer, Long, Real, Byte and blocking on 8 bit boundaries.

## 4.9 Parameters and INI options

### 4.9.1 Standard Parameters

| | | |
|---|---|---|
| Block | 256 | |
| Delay | 0 | (Citect V5 onwards does not require a delay in the driver) |
| MaxPending | 32 | |
| Polltime | 0 | |
| Timeout | 3000 | (Milliseconds) |
| Retry | 1 | |
| WatchTime | 75 (Seconds) | |

### 5.9.2 Driver Specific Parameters

General driver parameters - [moscad]

| Parameter | Default | Allowable values | Description |
|---|---|---|---|
| Channel | 7 | ?? | |
| Multiplexed | 1 | 0 or 1 | Enable redundancy at a unit level |
| MaxTimeouts | 3 | | Count |
| ConnTimeout | 15000 | | Ms. Connection timeout before a connection error is reported |
| TimeOutExt | 0 | | Global timeout extension, see per channel parameters |
| ReportUnknownAsAvail | 1 | 0 or 1 | If set to 1, then connection availability is TRUE if the units status is NOT OFFLINE. If set to 0, then connection availability is TRUE if the units status is ONLINE. |
| DatabaseLog | 0 | 0 or 1 | This parameter enables a database file of all received events (that have a Trend Tag or TimeStamped Alarm Tag defined) to be logged. This functionality requires the Devices form to be completed with an entry labelled RTULog. The Format in the form must correspond to that used in the Cicode program (refer to a sample project for this format). The file will reside in the location described by the File Name field entry in the Devices form. |

Parameters per port – [portname]

NB: Where “channel” exists, read this as the same as a “Port”

| Parameter | Default | Allowable values | Description |
|---|---|---|---|
| SetupFile | moscad.cfg | | Setupfile for the channel |
| GatewayMode | 1 | 1 or 2 | This parameter instructs the driver to treat the startup mode of each channel. If the GatewayMode is set to 1, the driver will put the Gateway into “Active” mode on Citect startup. If the gatewayMode is set to 2, the driver will put the Gateway into “Inactive” mode on Citect startup and this will prevents the gateway from data exchange with the RTUs. |

| | | | |
|---|---|---|---|
| Channel | 7 | ??? | Channel type setting or uses global value |
| Polled | 1 | 0 or 1 | 1 polled, 0 not polled. This is a configuration parameter which allows you to poll different tables in different RTU. It must be set in the moscad configuration file. |
| Multiplexed | 1 | 0 or 1 | Channel setting or uses global value |
| Tick | 500 | | Ms. Poll time if enabled. |
| NoPollPeriod | 10000 | | Ms. This is the INI parameter that controls the No Poll period in milliseconds. This setting will gurantee that during the NoPollPeriod there is no polling request sent to RTU on each channel. |
| DebugLevel | 0 | 0 or 1 | 1 for individual channel debug traces |
| Timeout | 3000 | 1 to 32767 | Ms. Message timeout |
| TimeoutExt | 0 | | Ms. Extension time on the timeout. |
| NoStatusCheck | 0 | 0 or 1 | 1 to disable status checks on coms to the gateway |

## 4.10 Configuration file

To work properly, the MOSCAD driver need information about the MOSCAD system and about the RTU types (communication table format). This information must be provided via an ASCII files, called configuration file. The configuration files ("<mdlc_par>.cfg", where mdlc_par is your file name) can be any name and anywhere in your local driver provided the path of this file is set up in Citect.ini file properly. To set up the path in Citect INI file, simply put SetupFile="Your File Path\\<mdlc_par>.cfg " under [MOSCAD] section in INI file. Note, use double back slash as delimiter. Please see the example in 5.10.3 for details.

<mdlc_par>.cfg - This file describes the format of the different existing RTU types (and future but currently non-existing RTU types) and type of each RTU in the MOSCAD system. There is a description of the communication tables format for each of the RTU types. The format defines the number of columns in a table, and the type of each column. A column can be either “Bit”, “Val”, “Float”, “C_Bit”, “TS1” or “TS2”; with Bit being for digitals, Val for 16 bit integers, Float for reals, C_Bit for eight columns of bits being compressed into one byte, and TS1 and TS2 used collectively for Time Stamping. Note that a TS1 column must be immediately followed by a TS2 column.
In this driver, there are two methods of timestamping data: individually or on a “per row” basis. To individually timestamp a data element, the data column in the table should be immediately followed by a TS1 and a TS2 column to hold the timestamp for that data element ( eg Bit TS1 TS2). In order to collectively timestamp an entire row, the first column should be of type TS1 and the second column should be of type TS2, all subsequent columns in the row will be timestamped by this value.

### 4.10.1 Format of configuration file

Format of line:

<label>:<id>:<parameter 1>;<parameter 2>[<arglist>];...<parameter n>:<text command>

where:

label is a text string without space and not starting with a number. eg Table0, Table_0.
id is a number 0-65536 used to identify the entity.

parameter 1.. n

for tables are:

| | |
|---|---|
| polled | this table will be polled. |
| rows <no> | is the number of row in the table. |
| coloumns <no> | is the number of coloumns. |

for units are:

| | |
|---|---|
| polled | this unit will be polled. |
| period <no ms> | is the number of ms between polls. |
| ratio <ratio> | is the number of event polls between integrity polls. |
| tables <label label ...> | is a list of table label defined before. |
| | the unit label configured in the unit. |

off for both tables and unit:
removes the unit/table from the list without causing syntax problems.

text command
for tables:
this command defines the column type in the table.

<type> <type> <type>
where:
type can be:
"real"
"discrete"
"analog"

for units:
this command defines the path and file name of the persistent cache of the unit

Note in standard unix form the line can traverse several line in the file by making the last charactor of the line prior to the <CR> a \.
Any charactors to the right of a # to the end of the line is comment.

### 4.10.2 Sample of a configuration file

```
# This is a sample of configuration file
#
# Table Configuration:
#
#<label> | : | <id> | : | < para1> | ; | < para 1> | ; | < para 1> | : | <Text Command>
#-----------------------------------------------------------------------------------------------------------------------
Table1    :  1    :  Polled    ;  Rows 4    ;  Columns 1    :  Discrete
Table2    :  2    :  Polled    ;  Rows 4    ;  Columns 1    :  analog
Table3    :  3    :  Polled    ;  Rows 4    ;  Columns 1    :  real
Table4    :  4    :            ;  Rows 4    ;  Columns 1    :  discrete real real
Table5    :  5    :            ;  Rows 4    ;  Columns 3    :  analog real real
Table6    :  6    :            ;  Rows 4    ;  Columns 3    :  real real real
Table7    :  7    :            ;  Rows 4    ;  Columns 3    :  real real discrete
Table8    :  8    :            ;  Rows 4    ;  Columns 3    :  real real discrete analog real
#
# RTU Configuration:
#
#<label>  | : | <id> | : | < para1> | ; | < para2> | ; | < para3>  | ; | < para4>  | : | <Text Command>
#-----------------------------------------------------------------------------------------------------------------------
---
RTU1      :  1    :  Ratio 2   ;  Polled    ;  Period 6000  ;  Tables  Table1\
                                                              Table2  Table3\
                                                              Table4 Table5\
                                                              Table6  Table7\
                                                              Table8          :  d:\tmp\rtu1.dat
RTU2      :  2    :  Ratio 2   ;  Polled    ;  Period 5000  ;  Tables  Table1\  :
```

Table2 Table3\
Table4 Table5\
Table6 Table7\
Table8

### 4.10.3 Sample of a CITECT.INI file

```
[MOSCAD]
!Setup Parameters for Citect Test Rig
!
!There are two discrete links used by Citect Test Rig
!       Rtus 1-2 on link1
!       Rtus 1 on link2
!

!Master Station or Gateway name
        Station=STATIONABC
! The Master station is the one responsible for polling the moscad gateways

!Citect timeout and retry
        Timeout=10000
        Retry = 2


!Burst, Send and/or Receive Channel
!       0 NORMAL - no Burst
!       1 SEND
!       2 RECEIVE
!       4 BURST
        Channel=7
! Channel = 7 sets the mode of the channel.
 Since it follows the station name definition and preceeds any specific channel.(e.g. STATIONABC)  It is the default channel mode for any following channel definitions, 7 = 4 + 2 + 1


!Other Parameters
!       DebugLevel
!       MaxTimeouts
!       ConnTimeout
!       RedundancyTick
!       Tick
        debugstr=PORT3_BOARD1 all

[STATIONABC0]

!Channels is not capable of fullduplex comms
        Multiplexed = 1
! Multiplexed This permits multiple requests on a single channel.
        Timeout=30000
        NoPollPeriod=2000

!Rtus 1 on link1 configuration
        SetupFile="d:\\CTDDK5\\VC6\\Drivers\\moscadip\\moscad1.cfg"

[STATIONABC1]

!Channels is not capable of fullduplex comms
        Multiplexed = 0

        Timeout=30000
        NoPollPeriod=2000
```

```
!Rtus 2 on link2 configuration
        SetupFile="d:\\CTDDK5\\VC6\\Drivers\\moscadip\\moscad2.cfg"

[STATIONABC2]

!Channels is not capable of fullduplex comms
        Multiplexed = 1

        Timeout=3000
        NoPollPeriod=7000

!Rtus 2 on link2 configuration
        SetupFile="d:\\CTDDK5\\VC6\\Drivers\\moscadip\\dnp.cfg"

============= End of INI -------------------------------
```

Extra notes

Polling definitions are defined in cfg. The cfg captures the MOSCAD unit definitions and table definitions. This parameter format is not supported by "ini" style data entry where a parameter occupies one line.

To have different tables on the same unit polled at different frequencies is not supported. The Full (integrity) poll, Change of State (COS) Poll and burst modes allow for optimising through put.
Note: All RTU can be in one .cfg file as specified by the MOSCAD(user spec).doc. You only need use different setup files (*.cfg) if you need to have multiple channels eg

```
[MOSCAD]

station = BASE

setup = "...defaultChannel.cfg"

[BASE0]

setup = "... firstChannel.cfg"

[BASE1]

setup = "... secondChannel.cfg"
```

where the intention is to use firstChannel to get to data for units, UNIT1 - UNIT20 say, and also use secondChannel to get to data for units, UNIT21 through UNIT30 say. So the idea is flexibility. However you can use one one default configuration file, as for defaultChannel.cfg in the above example.
Cache and scheduling sections in the IO Device form apply only to IOServer caching. Not the drivers.
Please be aware that the MOSCAD driver is a Front End / Back End (FE/BE) driver. Which means that there is an internal driver cache - separate from the IOServer cache.
Note: The parameters in the .cfg file pertain to the BE of the driver. I.e. the driver has a back end polling engine which will perform the regular polling tasks and processing of burst data regardless of data demand from the IOServer.
4. The channel will transmit data in one of the following means. Full (Integrity) Poll, COS Poll and burst modes. The combination of these will allow the engineer to optimise the bandwidth/throughput for the channel.

## 4.11 Using Gateway Redundancy

To use Gateway redundancy, the user must set the Gateway mode parameter in the Citect.ini file:

Eg.

```
[MOSCAD]
Station=CitectChannel

 [CitectChannel0]
GatewayMode=1          !treat gateway as “active” mode

[CitectChannel1]
GatewayMode=2          !treat gateway as “inactive” mode
```

[Refer to section 5.9.2 Driver Specific Parameters for details of the different startup modes]
[Refer to section 2.5.3.2 Gateway Mode of Operation Configuration for details]

## 4.12 Remapping

Remapping is not required or supported.

## 4.13 Hints and Tips

The use of the MOSCAD system with this driver provides a lot of flexibilty, but requires a knowledge of the MOSCAD system. It is assumed that the user of this driver is familiar with MOSCAD systems and the configuration of MOSCAD RTU’s. It is strongly recommended that the user reviews all of the information provided here before developing a project with this driver.

If the Syslog is reporting “Error: Driver is not responding”, then the driver may be unable to establish a TCP/IP connection. This is due to the driver being held up by the TCP/IP timeout, thus preventing the driver from responding back to Citect within its own timeout, causing Citect to issue an error.
Ensure that IP address has been set correctly in Citect and Gateway, and that all equipment is connected correctly.

Ensure that the configuration file <MDLC_Pra>.cfg have been set correctly as explained in Section 5.10, and that they have been included in the working Bin directory of Citect. The driver is dependant on these files being set correctly in order for it to correctly communicate with the RTU. Incorrect settings on these files will generate errors in the Syslog.

The API states that a user table configured in the RTU for use by Citect, should not have a discrete column as the first and last column, except if the table contains only discrete columns or only one column.

The driver will support the built in gateway redundancy provided by the MDLC API. From the users point of view this means that a backup gateway can be configured in the ports form, for the primary gateway, as a special option. A backup gateway should not be entered as a separate ports form entry.

There is a maximum of 250 RTUs supported per gateway.

✍ Radio transmission rates are typically very slow. In addition, RTUs are capable of independently “bursting” unsolicited data back to the gateway. If the system is not setup correctly, comms traffic may be saturated to the point that collisions reduce effective throughput to a trickle. This will also generate numerous errors and other complications. It must be verified that the timing of the comms system has been setup adequately before trying to faultfind any driver behaviour.

✍ When using redundancy across two IO Servers, the standby units will show their status on the devices form as “standby” when they are available (regardless of whether they are actually in standby mode or active). Use the “client using” field to determine if they are active.

**Setup Hints**

✍✍ The critical RTU must be the first device in the IO Devices form.

- A single critical RTU must be defined in the cfg setup file for the primary (ie all but one set to "noncritical"). The cfg setup file for the secondary must not contain any critical RTUs (ie all set to "noncritical").
- The IO Server holding the primary devices should have a "*GatewayMode*" INI parameter set to "1" in the INI section representing the first port. The IO Server holding the standby devices must have a "GatewayMode" INI parameter set to "2" in the INI section representing the first port. For example, if the INI files define the "*Station*" as "[MOSCAD] Station=Gamma", then the primary should have an INI section labelled [Gamma0] with a parameter "GatewayMode=1". Similarly, the secondary should also have an INI section labelled [Gamma0] but with a parameter "GatewayMode=2".
- Both IO Servers must have a " Multiplexed " INI parameter set to "0" in the INI section representing the first port. This indicates that both channels are sharing the physical communication link.
- The driver must be configured to provide adequate radio idle time (ie padding between polls). If the RTUs have not been configured to "burst" data back to the gateway, then only the standby needs to separate the polls to allow for the primary to perform its health check on the critical RTU. If "bursting" is used, then polling settings for both the primary and standby must allow enough radio silence for this to occur.
- Each gateway to be configured for "standalone" mode and each to have unique SiteIds.
- Each gateway must have its "health check" flag disabled.
- Each RTU has to be configured to detect a broadcast from the driver and to redirect its "burst" target to the relevant SiteId. This is required as only one gateway is active at a time, and the RTUs must burst their data back to the correct gateway. This is only a minor amount of RTU configuration. Refer to "Overview of redundancy using two standalone gateways".
- The Citect project will require cicode that broadcasts instructions to the RTU, causing them to redirect to a new burst target. This is required for failover. It must also periodically broadcast to cover the case of an RTU that may have been offline at change over. Tom Sabitzer, of the Lieden office, has written Cicode that provides this functionality. It has been tested exhaustively during the FAT, and is part of the solution for the Getronics project. Contact Tom for a copy of this cicode (royalties may apply!!).
- Any cicode commands issuing tag reads or tag writes should check the status virtual tag to ensure the device is available before actually issuing a command.
- It is recommended that the driver be set with 0 retries due to the reasons described in "Issues".
- The "Timeout" parameter must be set large enough to cater for the indeterminate nature of the gateway (as described in "Issues"). If required use the additional "TimeoutExt" parameter.
- The "Watchtime" parameter, for the IO Server holding the primary devices, must be set large enough so that the regular healthcheck of the critical RTU (when the channel is offline) will not interfere with the polling operation of the standby. A typical example may be 180 seconds.
- If writing application cicode that will issue commands resulting in physical communication to devices, ensure that the command rate is throttled to prevent swamping the diver with requests. The polling regime will also have to be tuned to free up enough idle time if this will occur on a permanent basis. Due to the slow nature of the physical comms, providing too high a vale of ""MaxPending" will run the risk of the driver buffering up so many requests that they can not all be processed for the IO Server in time. This will result in the IO Server deducing that the driver has ceased functioning and it will be reset. However, not having a high enough value will result in a bottleneck as the IO Server will have all of its requests tied up waiting for physical comms, with no requests spare to read values from cache. This will cause very strange behaviour that is peculiar to RTU comms (first observed in Perth on the Hunter WaterTech DNP project). The page update will freeze and the client may actually #COM its values even though the driver and IO Server are not reporting any errors!!!

# 5 Appendix

## 5.1 Notes from Field Work for version 3.2.x.x

### 5.1.1 Driver Issues

?? When the driver is communicating with the gateway it will start its timeout timer on transmission to the gateway. If the driver does not receive a response to that request within the timeout period, then it will issue a retry (if retries requested) and if all retries are exhausted then the request will be aborted. If this sequence happens in succession for n times (where 'n' is defined by the INI parameter "MaxTimeouts" which defaults to 3), then gateway is assumed to be uncontactable. The system will then fail over to the standby channel. The complication is that in addition to the gateway buffering the requests, its timeout timer is paused whenever the transmission path is busy. As the gateway can be configured for retries, the timeout for the retry may also be paused. The RTUs can also be set to "burst" (unsolicited response) data back to the gateway, and hence create additional comms traffic. As a consequence, the actual valid time for response is indeterminate. This is a very awkward situation as it makes it very difficult to determine if the gateway has failed or if there are just extended delays due to heavy comms usage. As the gateway should always return a response to every request, it should return an error code to indicate that it was unable to contact the RTU. The driver then detects and acts on that error code. However, due to the complexities just described, the driver may timeout before it receives the gateways response. The system should be setup to avoid this – gaps in the polling cycle to cater for "bursts" from the RTU and a timeout period much longer than the timeout sequence configured in the gateway. As a further note, if the driver is configured with retries, then it may repeat the request just prior to the gateways response, and hence the comms problem may "snowball" out of control.

?? If a non-critical RTU is offline and the communication link is overly busy, the phenomenon just described will result in the failure to detect that it is actually offline. This could be rectified by introducing timeout tallies for each unit and flagging the device as unavailable after "MaxTimeouts" successive failures. However, the system should be setup so that these erroneous timeouts do not occur.

✍ If there are errors in the cfg setup file, the unit may fail initialisation and cause the standby unit to become active. The cfg file must be corrected, as it is an invalid state to allow units to be mixed across both channels. Other errors may also fail initialisation of the unit and result with the same problem.

✍ Some INI parameters are only available in the ports INI section – they will be ignored if they are in the [MOSCAD] section.

?? Mechanism to pad radio silence between polls using the "NoPollPeriod" and "ChannelScanDelay" INI parameters needs further testing to verify its operation. As a separate note the code associated with ChannelScanDelay (tagged with "//DDK ELECCOM") may not be tightly integrated into the driver.

?? Cache persistence in the driver is currently only supported for standalone usage at startup. This results in redundancy failover and swapback not being bumpless with the display of incorrect values and the probable retriggering of alarms. The system will have synchronised itself with the field after a polling cycle. This problem was investigated on site but an effective solution was not forthcoming. This issue needs to be resolved.

### 5.1.2 Additional information regarding gateway operation

The following is an extract from an email of David Hassan (Motorola) to Getronics.
*(These details did not surface in **any** of our previous correspondence with Motorola)*

Q1: Is the behaviour, that after having downloaded the siteconfig into the

Gateway, the gateway initiates communication caused by the (default) setting "send broadcast after restart" and is it needed or can we turn it off?

A1: Upon Restart the IP Gateway sends a test message to each RTU defined in its Sites Table. It sends it only once after restart. This feature cannot be blocked. The IP Gateway retries this message to any RTU which does not respond to it. The default time out is around 1 minute. Thus if you have many not connected RTU's it will take a long time to finish all restart test messages. For the time being do not define in the IP Gateway Sites table, RTU's which are not connected in your test area.

Q2: It seems that the gateway is periodically sending data (once in a while communication takes place). what is causing the periodically sending of data by the gateway? Is it caused by the absense of Scada and the configured "startup-mode" as standalone. Also when I have only 3 RTU's configured in the sitetable (and all 3 are powered on) the periodical sending of data continues.

A2: The periodic message transmitted by the IP Gateway is an "Health Check" sent to each RTU every "Primary Link Validity" parameter defined in the IP Gateway Sites table for each RTU. This feature can be blocked in the IP Gateway Site Configuration (Advanced > message-oriented service > MORE (button) > Health-check support = NO)

Q3: Using a different configuration, startup-mode = redundant GW1, it looks like communication with the RTU's (setting and getting time, using the ip-gateway toolbox) is not possible. The gateway does not send any data. Do I have to make another configuration step I order to make it work? Or is it not working because Scada is missing?

A3: When the Gateway is in Secondary mode it does not activate its radio port. Thus it does not communicate with any RTU. If you want to use the Toolbox with the Gateway, when there is no Scada, you have to configure the IP Gateway as a Stand Alone in Start Up mode (see item 3 above)

### 5.1.3 Comments on code changes to the generic version (“AS31”)

- ?? Attempted to provide driver cache redundant persistence, however, this would not function consistently on failover to standby for reasons that could not be determined. It was functioning on cross over back to the primary. The current code will write to disk all cache values pertaining to a unit, every time that unit is polled. Hopefully this will only need to happen for the Standby, as in theory the primary should be able to be written to disk before swap over, as we should know ahead of time when this will occur. THIS CODE NEEDS TO COMPLETED, OR THE UNIT SHOULD GO TO #COM ON FAIL OVER.
- ?? The “multiplex” INI parameter is used to distinguish between “unit level” and “channel level” redundancy.
- ?? Critical unit functionality is available regardless of whether “multiplex” is set or not.
- ?? Comments have been placed in code for the relevant areas requiring change if implementing 4 way redundancy instead of just the current 2 way redundancy. However, in order for this to be achieved a method must be devised such that the first INIT_UNIT on the first standby path reads the status of its critical unit. Each path must have an independent critical unit so that one unit failing will not be interpreted as multiple channels failing. All non-critical units must always return the status of the critical unit to avoid being swapped to another channel independently. Hence the first INIT_UNIT must get the status of the critical unit, not its own status. This is currently worked around by the requirement that the critical RTU must always be the first device in the IO Devices form, and hence is the first INIT_UNIT. Also note that the last path must never have a critical unit, as it has no where to fail to!
- ?? The “NoStatusCheck” INI parameter causes the driver to only do a status check of the gateway instead of communicating with the actual RTU. This is only of consequence if “multiplex” is set, as a non-multiplexed (ie shared) channel will always only test the gateway for its STATUS_UNIT health checks.

?? A new INI parameter (for the [MOSCAD] section) has been introduced. “ReportUnknownAsAvail” can be used to determine whether the unknown state of a non-critical RTU will be reported as available or unavailable by the available virtual tag. The state of a non-critical RTU is unknown between startup and first poll and between failover and first poll.

### 5.1.4 Status of driver version

The driver was placed through all of the tests listed previously, to ensure that it would at least work with Getronics comms architecture. Despite the number of radical changes to the driver, only one test failed! This problem should be easily repeatable in Sydney and only actually requires a single RTU and gateway to reproduce. The problem occurs when the IO Server is started while the critical RTU can not be contacted (ie base radio off or critical RTU disconnected). Refer to comments and syslog in following section. The driver still has to be verified for use without redundancy (however, this was effectively tested as a subset of the redundancy tests). The driver needs to mainly be tested with multiplex set to one (ie usage over a comms architecture that supports “unit level” redundancy). From the brief usage I saw of the driver in this mode, there is a definite need to put a delay before first commencing polling to allow the health checks for all INIT_UNITS to proceed without the polling comms getting in the way. I also noted quite a few "data not yet valid” errors, but did not have time to chase these up. The real behaviour of the system (as the driver is only part of a highly reactive system) can only be determined when it is tested in a realistic comms architecture. This will either mean borrowing another gateway, a few more RTUs and radio gear or testing on site.

### 5.1.5 Possible Future work

Detect loss of channel via loss of Multiple critical RTUs. Detect return of channel via return of any 1 critical RTU
Detect loss of channel via loss of any ‘n’ consecutive RTUs.
Provide an option such that if an RTU can not be contacted on the final standby channel, of a multiplexed channel, then the driver does not issue a DRIVER_UNIT_OFFLINE, but does update the available virtual tag for that unit.
Provide 4 way redundancy between IO Servers and gateways.

### 5.1.6 Notes on temporary “freeze” at startup when critical RTU not contactable

Log of startup of AS31 with radio disconnected - Citect was "lockedup" for approximately a watchtime.
Unsure how the driver is making the IO Server "lockup" - I doubt it is failing to return processing time to Citect....
Note: timeout is actually 60 seconds (via additional INI parameter)

N.B. Transmit is at.....
Sat Sep 28 15:50:39 2002 168:10:29.172 Transmit(0): DATETIME 1-0 Length 52
00 01 00 00 00 00 00 01 00 00 00 00 00 00 00 00 ................
00 34 00 01 00 00 04 01 00 00 00 00 00 00 00 00 .4..............
00 10 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ................
00 00 00 00 ....

and timeout is at .....
Sat Sep 28 15:53:46 2002 168:13:35.719 Timeout - Tune timeout settings and radio usage. Length 52
00 01 00 00 00 00 00 01 00 00 00 00 00 00 00 00 ................
00 34 00 01 00 00 04 01 00 00 00 00 00 00 00 00 .4..............
00 10 00 00 00 01 00 00 00 00 00 00 00 00 00 00 ................
00 00 00 00 ....

A similar thing was happening when the crtical RTU was offline, this turned out to be the code in

processrcvdpacket recharging the timeout on every burst received. The radio is off so there can not be any burst data, but there is still code in there that is recharging the timeout for reasons unknown. This is the first place to look - it is unknown what the gateway is doing when the radio is disconnected.

I just realised.... the gateway should have responded, so the driver should not have timed out. It looks like the gateway may be getting confused, but we must still handle this situation.

This is not a major issue, as the standby will take over immediately - the IO Server must not respond to the client.....

D.R. 28/9/2002

```
Sat Sep 28 15:49:58 2002 Moscad Driver v3.01.03.061 AS31 driver loaded from C:\Citect\Bin\MOSCAD.DLL
 DLL modification time is 11:57:44 28/9/2002, file size is 73804 bytes
 Driver base address 0x61010000, Common Parameters:
 MaxPending=8 Block=256 Retry=0 Delay=100 Timeout=0 PollTime=0 WatchTime=180000


Sat Sep 28 15:49:58 2002 Disk Driver v3.00.09.000  driver loaded from C:\Citect\Bin\DISKDRV.DLL
 DLL modification time is 12:46:48 2/4/2002, file size is 42688 bytes
 Driver base address 0x60e00000, Common Parameters:
 MaxPending=20 Block=80 Retry=2 Delay=5 Timeout=2000 PollTime=1000 WatchTime=30000


Sat Sep 28 15:49:58 2002 Alarm States PLC, Version 4.0 driver loaded from C:\Citect\Bin\Alarm.DLL
 DLL modification time is 12:43:20 2/4/2002, file size is 38288 bytes
 Driver base address 0x60df0000, Common Parameters:
 MaxPending=4 Block=80 Retry=0 Delay=0 Timeout=1000 PollTime=0 WatchTime=30000

Sat Sep 28 15:50:39 2002 168:10:28.710 Citect Startup 5.41 SpkA
Sat Sep 28 15:50:39 2002 168:10:28.713 Transmit(0):       CONNECT Length 45
00 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00      ................
00 2D 00 01 00 00 00 00 01 00 00 00 00 00 00 00 00      .-..............
00 09 00 00 07 55 72 75 6D 71 69 30 00                 .....Urumqi0.
Sat Sep 28 15:50:39 2002 168:10:28.913 Received(0):       CONNECT Length 43
00 01 00 00 00 00 00 00 00 00 00 BD 00 00 00 01      ................
00 2B 00 01 80 00 00 01 09 01 00 00 00 00 00 00      .+..............
00 07 00 00 00 09 07 00 00 00 BD                       ...........
Sat Sep 28 15:50:39 2002 168:10:29.172 Transmit(0):       DATETIME 1-0 Length 52
00 01 00 00 00 00 00 01 00 00 00 00 00 00 00 00      ................
00 34 00 01 00 00 00 04 01 00 00 00 00 00 00 00 00      .4..............
00 10 00 00 00 01 00 00 00 00 00 00 00 00 00 00      ................
00 00 00 00                                            ....
Sat Sep 28 15:50:39 2002 168:10:29.414 PollService : Channel=0 SiteId=2 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:40 2002 168:10:29.915 PollService : Channel=0 SiteId=3 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:40 2002 168:10:30.415 PollService : Channel=0 SiteId=4 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:41 2002 168:10:30.915 PollService : Channel=0 SiteId=5 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:41 2002 168:10:31.416 PollService : Channel=0 SiteId=6 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:42 2002 168:10:31.917 PollService : Channel=0 SiteId=7 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:42 2002 168:10:32.417 PollService : Channel=0 SiteId=8 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:43 2002 168:10:32.918 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:43 2002 168:10:33.419 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:44 2002 168:10:33.920 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:44 2002 168:10:34.420 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:45 2002 168:10:34.921 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:45 2002 168:10:35.421 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:46 2002 168:10:35.922 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:46 2002 168:10:36.423 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:47 2002 168:10:36.924 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:47 2002 168:10:37.425 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:48 2002 168:10:37.927 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:48 2002 168:10:38.427 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:49 2002 168:10:38.927 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:49 2002 168:10:39.428 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:50 2002 168:10:39.928 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:50 2002 168:10:40.429 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:51 2002 168:10:40.930 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:51 2002 168:10:41.431 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:52 2002 168:10:41.931 PollService : Channel=0 SiteId=1 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:52 2002 168:10:42.431 PollService : Channel=0 SiteId=2 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:53 2002 168:10:42.932 PollService : Channel=0 SiteId=3 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:53 2002 168:10:43.432 PollService : Channel=0 SiteId=4 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:54 2002 168:10:43.933 PollService : Channel=0 SiteId=5 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:54 2002 168:10:44.434 PollService : Channel=0 SiteId=6 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:55 2002 168:10:44.935 PollService : Channel=0 SiteId=7 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:55 2002 168:10:45.435 PollService : Channel=0 SiteId=8 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:56 2002 168:10:45.936 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:56 2002 168:10:46.437 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:57 2002 168:10:46.937 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:57 2002 168:10:47.437 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:58 2002 168:10:47.937 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:58 2002 168:10:48.438 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:59 2002 168:10:48.938 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:50:59 2002 168:10:49.439 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:00 2002 168:10:49.944 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:00 2002 168:10:50.448 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:01 2002 168:10:50.955 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:01 2002 168:10:51.463 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:02 2002 168:10:51.970 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:02 2002 168:10:52.475 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:03 2002 168:10:52.981 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:03 2002 168:10:53.485 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:04 2002 168:10:53.987 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
```

```
Sat Sep 28 15:51:04 2002 168:10:54.490 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:05 2002 168:10:54.994 PollService : Channel=0 SiteId=1 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:05 2002 168:10:55.497 PollService : Channel=0 SiteId=2 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:06 2002 168:10:56.003 PollService : Channel=0 SiteId=3 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:07 2002 168:10:56.512 PollService : Channel=0 SiteId=4 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:07 2002 168:10:57.017 PollService : Channel=0 SiteId=5 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:08 2002 168:10:57.525 PollService : Channel=0 SiteId=6 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:08 2002 168:10:58.034 PollService : Channel=0 SiteId=7 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:09 2002 168:10:58.535 PollService : Channel=0 SiteId=8 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:09 2002 168:10:59.037 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:51:09 2002 168:10:59.541 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:46 2002 168:13:35.719 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:46 2002 168:13:35.719 Timeout - Tune timeout settings and radio usage. Length 52
00 01 00 00 00 00 00 01 00 00 00 00 00 00 00 00      ................
00 34 00 01 00 00 04 01 00 00 00 00 00 00 00 00      .4..............
00 10 00 00 00 01 00 00 00 00 00 00 00 00 00 00      ................
00 00 00 00                                          ....
Sat Sep 28 15:53:46 2002 168:13:35.719 Error: Channel offline, cannot talk
  UINIT    0015     Urumqi0       RTU1                        16
  Generic 000021 Driver 00000020 (0x00000014)
Sat Sep 28 15:53:46 2002 168:13:36.382 Transmit(0):     GET STATUS 2 Length 36
00 01 00 00 00 00 00 02 00 00 00 00 00 00 00 00      ................
00 24 00 01 00 00 00 09 00 00 00 00 00 00 00 00      .$..............
00 00 00 00                                          ....
Sat Sep 28 15:53:46 2002 168:13:36.382 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:46 2002 168:13:36.399 Error in DataBlock Header during INIT_UNIT 0x0 - Channel:(0) Length 0
Sat Sep 28 15:53:46 2002 168:13:36.399 Received(0):     GET STATUS Length 40
00 01 00 00 09 00 00 01 00 00 00 BD 00 00 00 01      ................
00 28 00 01 80 00 00 09 09 00 00 02 00 00 00 00      .(..............
00 04 00 00 00 00 00 01                              ........
Sat Sep 28 15:53:46 2002 168:13:36.475 Error: Data not yet valid
  UINIT    0019     Urumqi0       RTU2                        16
  Generic 000025 Driver 00004155 (0x0000103b)
Sat Sep 28 15:53:46 2002 168:13:36.510 Transmit(0):     GET STATUS 3 Length 36
00 01 00 00 00 00 00 03 00 00 00 00 00 00 00 00      ................
00 24 00 01 00 00 00 09 00 00 00 00 00 00 00 00      .$..............
00 00 00 00                                          ....
Sat Sep 28 15:53:46 2002 168:13:36.525 Received(0):     GET STATUS Length 40
00 01 00 00 09 00 00 02 00 00 00 BD 00 00 00 01      ................
00 28 00 01 80 00 00 09 09 00 00 03 00 00 00 00      .(..............
00 04 00 00 00 00 00 01                              ........
Sat Sep 28 15:53:46 2002 168:13:36.571 Error: Unit offline
  UINIT    000c     Urumqi0       RTU3                        16
  Generic 000012 Driver 00000023 (0x00000017)
Sat Sep 28 15:53:47 2002 168:13:36.640 Transmit(0):     GET STATUS 4 Length 36
00 01 00 00 00 00 00 04 00 00 00 00 00 00 00 00      ................
00 24 00 01 00 00 00 09 00 00 00 00 00 00 00 00      .$..............
00 00 00 00                                          ....
Sat Sep 28 15:53:47 2002 168:13:36.656 Received(0):     GET STATUS Length 40
00 01 00 00 09 00 00 03 00 00 00 BD 00 00 00 01      ................
00 28 00 01 80 00 00 09 09 00 00 04 00 00 00 00      .(..............
00 04 00 00 00 00 00 01                              ........
Sat Sep 28 15:53:47 2002 168:13:36.656 Error: Unit offline
  UINIT    000c     Urumqi0       RTU4                        16
  Generic 000012 Driver 00000023 (0x00000017)
Sat Sep 28 15:53:47 2002 168:13:36.805 Transmit(0):     GET STATUS 5 Length 36
00 01 00 00 00 00 00 05 00 00 00 00 00 00 00 00      ................
00 24 00 01 00 00 00 09 00 00 00 00 00 00 00 00      .$..............
00 00 00 00                                          ....
Sat Sep 28 15:53:47 2002 168:13:36.817 Received(0):     GET STATUS Length 40
00 01 00 00 09 00 00 04 00 00 00 BD 00 00 00 01      ................
00 28 00 01 80 00 00 09 09 00 00 05 00 00 00 00      .(..............
00 04 00 00 00 00 00 01                              ........
Sat Sep 28 15:53:47 2002 168:13:36.818 Error: Unit offline
  UINIT    000c     Urumqi0       RTU5                        16
  Generic 000012 Driver 00000023 (0x00000017)
Sat Sep 28 15:53:47 2002 168:13:36.895 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:47 2002 168:13:36.946 Transmit(0):     GET STATUS 6 Length 36
00 01 00 00 00 00 00 06 00 00 00 00 00 00 00 00      ................
00 24 00 01 00 00 00 09 00 00 00 00 00 00 00 00      .$..............
00 00 00 00                                          ....
Sat Sep 28 15:53:47 2002 168:13:36.969 Received(0):     GET STATUS Length 40
00 01 00 00 09 00 00 05 00 00 00 BD 00 00 00 01      ................
00 28 00 01 80 00 00 09 09 00 00 06 00 00 00 00      .(..............
00 04 00 00 00 00 00 01                              ........
Sat Sep 28 15:53:47 2002 168:13:37.019 Error: Unit offline
  UINIT    000c     Urumqi0       RTU6                        16
  Generic 000012 Driver 00000023 (0x00000017)
Sat Sep 28 15:53:47 2002 168:13:37.089 Transmit(0):     GET STATUS 7 Length 36
00 01 00 00 00 00 00 07 00 00 00 00 00 00 00 00      ................
00 24 00 01 00 00 00 09 00 00 00 00 00 00 00 00      .$..............
00 00 00 00                                          ....
Sat Sep 28 15:53:47 2002 168:13:37.106 Received(0):     GET STATUS Length 40
00 01 00 00 09 00 00 06 00 00 00 BD 00 00 00 01      ................
00 28 00 01 80 00 00 09 09 00 00 07 00 00 00 00      .(..............
00 04 00 00 00 00 00 01                              ........
Sat Sep 28 15:53:47 2002 168:13:37.157 Error: Unit offline
  UINIT    000c     Urumqi0       RTU7                        16
  Generic 000012 Driver 00000023 (0x00000017)
Sat Sep 28 15:53:47 2002 168:13:37.208 Transmit(0):     GET STATUS 8 Length 36
00 01 00 00 00 00 00 08 00 00 00 00 00 00 00 00      ................
00 24 00 01 00 00 00 09 00 00 00 00 00 00 00 00      .$..............
00 00 00 00                                          ....
Sat Sep 28 15:53:47 2002 168:13:37.222 Received(0):     GET STATUS Length 40
00 01 00 00 09 00 00 07 00 00 00 BD 00 00 00 01      ................
00 28 00 01 80 00 00 09 09 00 00 08 00 00 00 00      .(..............
00 04 00 00 00 00 00 01                              ........
Sat Sep 28 15:53:47 2002 168:13:37.222 Error: Unit offline
  UINIT    000c     Urumqi0       RTU8                        16
  Generic 000012 Driver 00000023 (0x00000017)
```

```
Sat Sep 28 15:53:47 2002 168:13:37.445 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:48 2002 168:13:37.979 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:49 2002 168:13:38.498 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:49 2002 168:13:39.032 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:50 2002 168:13:39.551 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:50 2002 168:13:40.061 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:51 2002 168:13:40.587 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:51 2002 168:13:41.115 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:52 2002 168:13:41.642 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:52 2002 168:13:42.158 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:53 2002 168:13:42.696 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:53 2002 168:13:43.224 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:54 2002 168:13:43.752 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:54 2002 168:13:44.275 PollService : Channel=0 SiteId=1 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:55 2002 168:13:44.806 PollService : Channel=0 SiteId=2 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:55 2002 168:13:45.345 PollService : Channel=0 SiteId=3 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:56 2002 168:13:45.876 PollService : Channel=0 SiteId=4 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:56 2002 168:13:46.387 PollService : Channel=0 SiteId=5 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:57 2002 168:13:46.918 PollService : Channel=0 SiteId=6 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:57 2002 168:13:47.443 PollService : Channel=0 SiteId=7 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:58 2002 168:13:47.974 PollService : Channel=0 SiteId=8 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:59 2002 168:13:48.495 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:53:59 2002 168:13:49.026 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:00 2002 168:13:49.549 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:00 2002 168:13:50.049 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:01 2002 168:13:50.550 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:01 2002 168:13:51.050 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:02 2002 168:13:51.550 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:02 2002 168:13:52.051 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:03 2002 168:13:52.552 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:03 2002 168:13:53.052 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:04 2002 168:13:53.552 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:04 2002 168:13:54.052 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:05 2002 168:13:54.553 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:05 2002 168:13:55.054 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:06 2002 168:13:55.554 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:06 2002 168:13:56.054 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:07 2002 168:13:56.554 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:07 2002 168:13:57.054 PollService : Channel=0 SiteId=0 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:08 2002 168:13:57.555 PollService : Channel=0 SiteId=1 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:08 2002 168:13:58.055 PollService : Channel=0 SiteId=2 Resource=0(2=Act/0=Inact) Polling=0 Length 0
Sat Sep 28 15:54:09 2002 168:13:58.556 PollService : Channel=0 SiteId=3 Resource=0(2=Act/0=Inact) Polling=0 Length 0
```

## 5.2 MOSCAD Driver Modification List for V30103002b1

### 5.2.1 C_BIT Problem

Compressed digital Value for 16Digital Input and two counter module (FRN1420A) or 16 digital out module (FRN1491A) that value need 16 bit word data range but old driver (3.00.00.001B2) can't read that value with following error : "Data not yet valid"

So I modified source file "Moscad_Message.cpp" line 2400 for multiple C_BIT counts.
If Table of Mdlc configuration is below configuration(ex1) then pDcb->Point.UnitCount value exceed tru->ushMaxRows value, so there's encounter data not yet valid error

Ex1)
Table6:4:Polled; Rows 5; Columns 8:analog analog analog analog analog analog analog analog

So I attached following lines.

```
//DDK ELECCOM START
if      ((pPoint->RawType != RDT_DIGITAL ) OR
         (((nRow + pPoint->UnitCount) / 8) > rtu->ushMaxRows*2))
//DDK ELECCOM END
```

and in ReadTable function I attached following lines (Line 2493 of Moscad_Message.cpp)
because of digital remapping data for C_BIT

### 5.2.2 C_BIT bit swap operation

In FRN1420A Module, first digital input relay matches Ann.nn.nn.15 and last (16th) digital input relay matches Ann.nn.nn.0. this means MSB and LSB swapped
So I attached digital relay swapping function(Line 2363 Moscad_Message.cpp),
and parameter of driver "**AnalogBitSwap**"

I also attached parameter "analogWordSwap" but I think that there's no useful this parameter.

### 5.2.3 Timeout Value Long

I changed this variable type from SHORT to ULONG, because customer want to set this value one or two minutes, that is to say over 32767 millisecond.

### 5.2.4 POLLING PROCESS OPERTION

When poll process of MOSCAD driver, data gathering in every NoPollPeriod is being processed, but each time there's only one station to communicate with MOSCAD driver. When Next polltime, MOSCAD Drvier communicate with another station.
That is to say if there're some station is connected with gateway, data update time of each station is 'NoPollPeriod * station number'.
So I modified source file to communicate with each station on one scan period.

So I attached following lines (Line 847 of Moscad_Get.cpp)

```
// for Next Station Polling
{
        UNIT_DB *pUnitTemp = pCp->pUnitToPoll;
      if (NextUnitToPoll(&(pUnitTemp), pCp->pUnitDataBase, pCp->nMaxUnit))
      {
            pCp->lNextPollTime = nChannelScanDelay;
      }
}
```

nChannelDelay is Citect.Ini value that I attached ( line 1945 of Moscad_Init.cpp)

nChannelScanDelay = (BOOLEAN)GetCitectIniLong(Constants.Name, "ChannelScanDelay", 0);

### 5.2.5 Channel Data Reading problem on start time

When MOSCAD driver is started, Channel of state (**ResourceControlState**)is changed by another routine.
There's some sample code on Line 288 of Moscad_Send.cpp
If (pCp->bPollInProgress &&
**(pCp->pUnitToPoll)->ResourceControlState == RESOURCES_GAINED**)

when Resource Gain state of unit is changed to REESOURCE_GAINED, this unit is not first station every time. This is problem. 'NextUnitToPoll' function can find next station but previous station. so previous station data will be collected on next polling time.
If NoPollPeriod is very long(more 5 minutes), citect graphic mimic cannot display this value.
This problem is conquered using cache file in moscad.cfg files
RTU1:100:ratio 2; Polled; Period 500; Tables Table5 Table6 Table14 : **d:\temp\rtu01.dat**